



Dr. M.G.R.
EDUCATIONAL AND RESEARCH INSTITUTE
DEEMED TO BE UNIVERSITY



University with Graded Autonomy Status

(An ISO 21001 : 2018 Certified Institution)

Periyar E.V.R. High Road, Maduravoyal, Chennai-95. Tamilnadu, India.

RECORD NOTEBOOK

INTRODUCTION OF

BIGDATA ANALYTICS LAB – (EBAI22ET2)

2024-2025(EVEN SEMESTER)

DEPARTMENT

Of

COMPUTER SCIENCE AND ENGINEERING

NAME : P.HARSHITHA

REGISTER NO : 231641101168

COURSE : B.TECH-CSE(DS)

YEAR/SEM/SEC : II/IV/BB



Dr. M.G.R.
EDUCATIONAL AND RESEARCH INSTITUTE
DEEMED TO BE UNIVERSITY



University with Graded Autonomy Status

(An ISO 21001 : 2018 Certified Institution)

Periyar E.V.R. High Road, Maduravoyal, Chennai-95. Tamilnadu, India.

BONAFIDE CERTIFICATE

Register No : 231641101168

Name of Lab : INTRODUCTION OF BIGDATA ANALYTICS
LAB-(EBAI22ET2)

Department : COMPUTER SCIENCE AND ENGINEERING (DS)

Certified that this is the bonafide record of work done by **P.HARSHITHA 231641101168** of II Year B.Tech(CSE-DS), Sec-‘E’ in the **INTRODUCTION OF BIGDATA ANALYTICS LAB (EBAI22ET2)** during the year 2024-2025.

Signature of Lab-in-Charge

Signature of Head of Dept

Submitted for the Practical Examination held on _____

Internal Examiner

External Examiner

INDEX

Exp. No.	DATE	Title	Page No.	STAFF SIGNATURE
1	01/04/2025	EXPLORATION OF THE HADOOP INSTALLATION	01-10	
2	29/04/2025	HADOOP IMPLEMENTATION OF FILE MANAGEMENT TASKS	11-16	
3	13/05/2025	IMPLEMENT OF MATRIX MULTIPLICATION WITH HADOOP MAP REDUCE	17-28	
4	27/05/2025	RUN A BASIC WORD COUNT REDUCE PROGRAM TO UNDERSTAND MAP REDUCE PARADIGM	29-37	
5	17/06/2025	IMPLEMENTATION OF KMEANS CLUSTERING USING MAP REDUCE	38-48	
6	24/06/2025	IMPLEMENTATION OF KMEANS CLUSTERING USING MAP REDUCE	49-66	

EXPLORATION OF INSTALLTION OF HADOOP

AIM: To install and setup Hadoop environment in windows

ALGORITHM:

STEP 1: To install Hadoop the primary task is to setup and install java environment

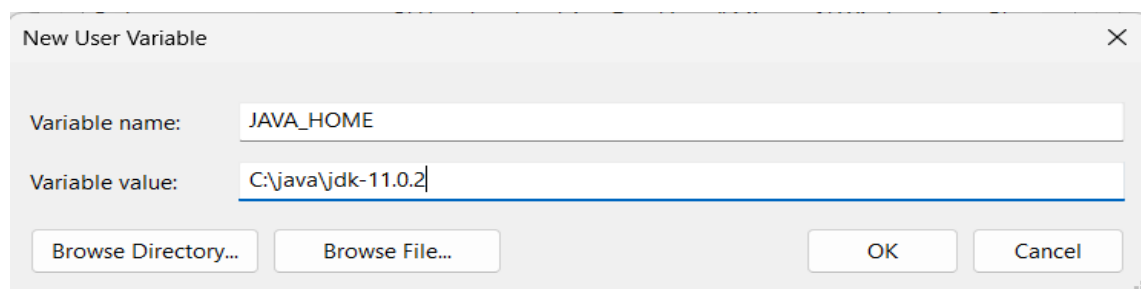
STEP 2: The java version that needed to be installed depends on the Hadoop's version. Here we are installing the latest version of Hadoop which is 3.3.0 which supports java version varying from 8-11(runtime only).

STEP 3: Use the following link to install java

<https://www.oracle.com/java/technologies/downloads/#java8-windows>

Linux	macOS	Solaris	Windows
Product/file description		File size	Download
x86 Installer		136.83 MB	 jdk-8u381-windows-i586.exe
x64 Installer		145.55 MB	 jdk-8u381-windows-x64.exe

STEP 4: After installing java setup, the java environment in environmental variables directing the bin folder inside the java folder (C:\java\jdk-11.0.2\bin) copy the path till bin folder and paste it in the environmental variable define the new path and add the bin folder location as **JAVA_HOME="C:\java\jdk11.0.2\bin"** and apply the changes



STEP 5: Now after setting up the java environment check the setup has been successfully set by using **java -version** command in your command prompt and it should display the version of java you have installed.

```
C:\Windows\System32>java -version
java version "1.8.0_381"
Java(TM) SE Runtime Environment (build 1.8.0_381-b09)
Java HotSpot(TM) 64-Bit Server VM (build 25.381-b09, mixed mode)
```

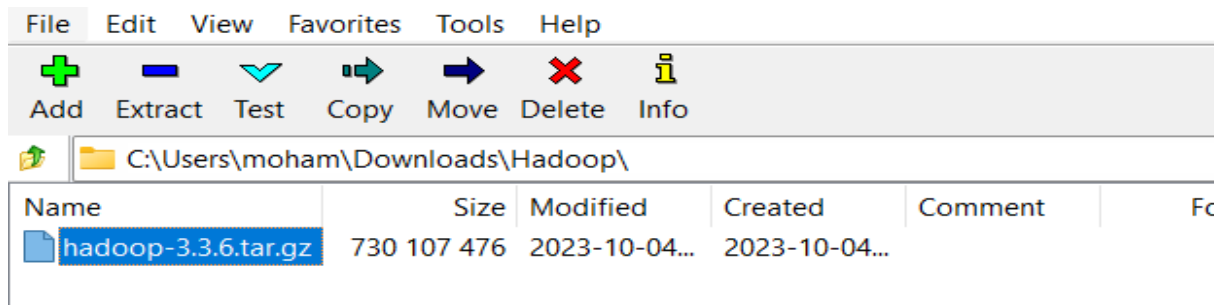
STEP 7: Hadoop is Unix distribution-based file with tar.gz extension we have to extract the file using the 7-zip manager which supports multiple formats follow this link to install 7-zip <https://7-zip.org/>

STEP 8: Now install the Notepad++ text editor which is further used to modify or edit the configuration file within Hadoop as per our requirement

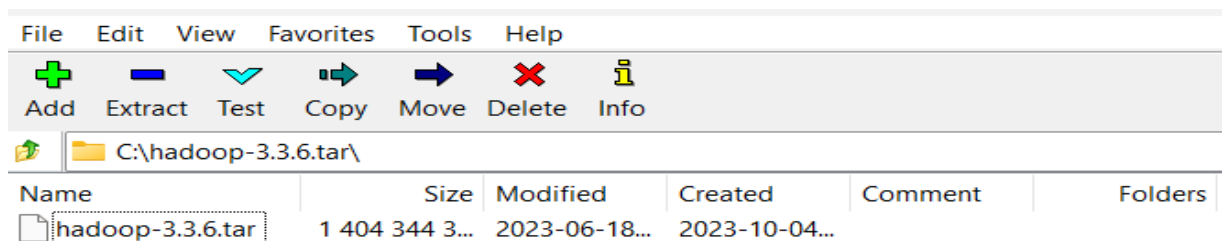
STEP 9: After installing and setting up all the required application install Hadoop from the official Apache Hadoop website <https://hadoop.apache.org/releases.html> download the binary download which can run directly without any need for compilation.

Version	Release date	Source download	Binary download	Release notes
3.3.6	2023 Jun 23	source (checksum signature)	binary (checksum signature) binary-aarch64 (checksum signature)	Announcement
3.2.4	2022 Jul 22	source (checksum signature)	binary (checksum signature)	Announcement
2.10.2	2022 May 31	source (checksum signature)	binary (checksum signature)	Announcement

STEP 10: Run 7-zip manager as administrator and navigate to the path where Hadoop is located for extract the compiled binary download of Hadoop



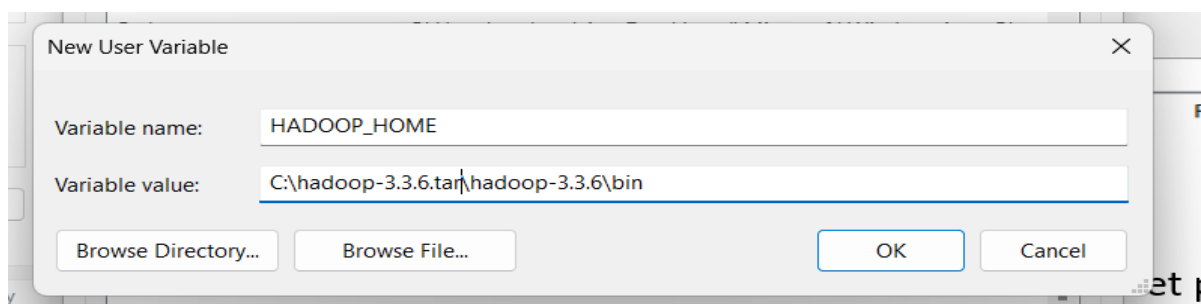
STEP 11: After doing the extraction process there is another compressed file with in the extracted file extract that as well.



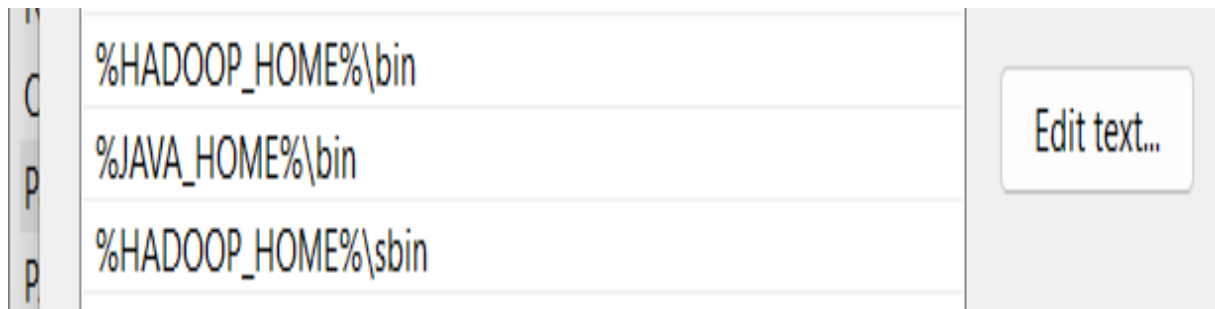
STEP 12: From the extracted folder replace the bin file with the reliable windows supported configured file here is the drive link to download the bin file

<https://drive.google.com/file/d/1kVhX9snOZ3oLUxDjh3AVI8fcRnEWAAE4/view>

STEP 13: Setup the Hadoop environment in environment variable and set path location as
HADOOP_HOME= "C:\hadoop-3.3.6.tar\hadoop-3.3.6\bin"



STEP 14: Add the Hadoop bin and sbin path location by editing the path. And add the bin, sbin location there



STEP 15: Now open etc folder inside the Hadoop folder and locate the file Hadoop-env.cmd and set the java home location

```
22 @rem remote nodes.
23
24 @rem The java implementation to use. Required.
25 set JAVA_HOME=%JAVA_HOME%
26 set JAVA_HOME=C:\java\java8
27
28 @rem The jsvc implementation to use. Jsvc is required to run secure datanodes.
29 @rem set JSVC_HOME=%JSVC_HOME%
30
31 @rem set HADOOP_CONF_DIR=
```

STEP 16: Edit the following configuration XML files core-site.xml, hdfs-site.xml, mapred-site.xml, yarn-site.xml are used to configure the behaviour of your Hadoop Cluster and save them.

STEP 17: Starting from core-site.xml edit it using notepad++ and following the program to configure.

PROGRAM (CORE-SITE.XML)

```
<configuration>

<property>

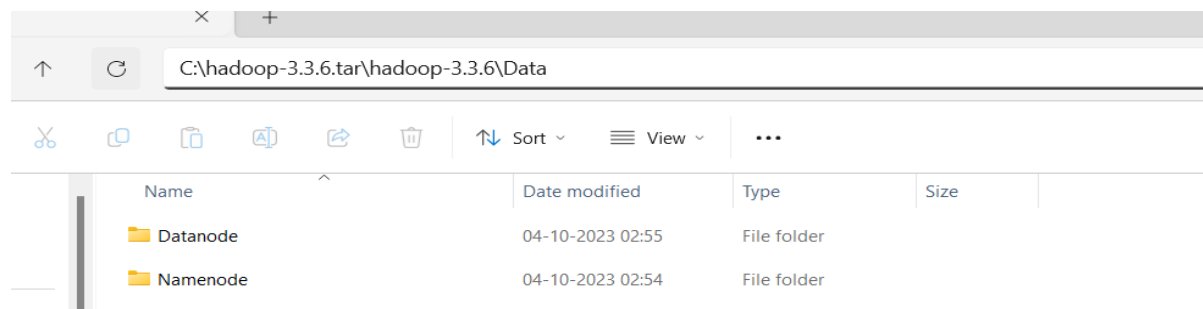
<name>fs.default.name</name>

<value>hdfs://localhost:9000</value>

</property>

</configuration>
```

STEP 18: To edit the hdfs-site.xml create Data folder and then within the Data folder create Namenode, Datanode which are used to manage and cluster flow data and log files.



STEP 19: Now open the hdfs-site.xml in notepad++ and add the following program

```
<configuration>
```

```
<property>
```

```
<name>dfs.replication</name>
```

```
<value>1</value>
```

```
</property>
```

```
<property>
```

```
<name>dfs.namenode.name.dir</name>
```

```
<value> C:/hadoop-3.3.6.tar/hadoop-3.3.6/Data/Datanode </value>
```

```
</property>
```

```
<property>
```

```
<name>dfs.datanode.data.dir</name>
```

```
<value>> C:/hadoop-3.3.6.tar/hadoop-3.3.6/Data/Datanode </value>
```

```
</property>
```

```
</configuration>
```

STEP 20: Edit mapred-site.xml file using notepad++ add this following program

```
<configuration>
```

```
<property>
```

```
<name>mapreduce.framework.name</name>
```

```
<value>yarn</value>
```

```
</property>
```

```
</configuration>
```

STEP 21: Edit the yarn-site.xml with following program and save it.

```
<configuration>
```

```
<property>
```

```
<name>yarn.nodemanager.aux-services</name>
```

```
<value>mapreduce_shuffle</value>
```

```
</property>
```

```
<property>
```

<name>yarn.nodemanager.auxservices.mapreduce.shuffle.class</name>

<value>org.apache.hadoop.mapred.ShuffleHandler</value>

</property>

</configuration>

STEP 22: Now save them and open command prompt as administrator and run the following command to **hdfs namenode -format** to format the contents of namenode

```
\current\fsimage.ckpt_000000000000000000 using no compression
2023-11-06 11:45:46,760 INFO namenode.FSImageFormatProtobuf: Image file C:\hadoop-3.3.0\data\namenode1290\current\fsimage.ckpt_000000000000000000 of size 400 bytes saved in 0 seconds .
2023-11-06 11:45:46,783 INFO namenode.NNStorageRetentionManager: Going to retain 1 images with txid >= 0
2023-11-06 11:45:46,794 INFO namenode.FSImage: FSImageSaver clean checkpoint: txid=0 when meet shutdown.
2023-11-06 11:45:46,795 INFO namenode.NameNode: SHUTDOWN_MSG:
/*****
SHUTDOWN_MSG: Shutting down NameNode at Azhar/127.0.0.1
*****/
C:\Windows\system32>
```

STEP 23: To check the daemons configured correctly open command prompt as administrator and run the following command's

hdfs namenode, hdfs Datanode, yarn nodemanager, yarn resourcemanager

hdfs namenode

```
C:\hadoop-3.3.0\sbin>hdfs namenode
2023-11-06 11:47:10,614 INFO namenode.NameNode: STARTUP_MSG:
/*****
STARTUP_MSG: Starting NameNode
STARTUP_MSG:  host = Azhar/127.0.0.1
STARTUP_MSG:  args = []
STARTUP_MSG:  version = 3.3.0
STARTUP_MSG:  classpath = C:\hadoop-3.3.0\etc\hadoop;C:\hadoop-3.3.0\share\hadoop\common;C:\hadoop-3.3.0\share\hadoop\common\lib\accessors-smart-1.2.jar;C:\hadoop-3.3.0\share\hadoop\common\lib\animal-sniffer-annotations-1.1
```

hdfs datanode

```
C:\Windows\system32>hdfs datanode
2023-11-06 11:51:47,958 INFO datanode.DataNode: STARTUP_MSG:
/*****
STARTUP_MSG: Starting DataNode
STARTUP_MSG:   host = Azhar/127.0.0.1
STARTUP_MSG:   args = []
STARTUP_MSG:   version = 3.3.0
STARTUP_MSG:   classpath = C:\hadoop-3.3.0\etc\hadoop;C:\hadoop-3.3.0\share\hadoop\common;C:\hadoop-3.3.0\share\hadoop\common\lib\accessors-smart-1.2.jar;C:\hadoop-3.3.0\share\hadoop\common\lib\animal-sniffer-annotations-1.17.jar;C:\hadoop-3.3.0\share\hadoop\common\lib\asm-5.0.4.jar;C:\hadoop-3.3.0\share\
```

yarn nodemanager

```
C:\Windows\system32>yarn nodemanager
2023-11-06 11:52:28,180 INFO nodemanager.NodeManager: STARTUP_MSG:
/*****
STARTUP_MSG: Starting NodeManager
STARTUP_MSG:   host = Azhar/127.0.0.1
STARTUP_MSG:   args = []
STARTUP_MSG:   version = 3.3.0
STARTUP_MSG:   classpath = C:\hadoop-3.3.0\etc\hadoop;C:\hadoop-3.3.0\etc\hadoop;C:\hadoop-3.3.0\etc\hadoop;C:\hadoop-3.3.0\share\hadoop\common;C:\hadoop-3.3.0\share\hadoop\common\lib\accessors-smart-1.2.jar;C:\hadoop-3.3.0\share\hadoop\common\lib\animal-sniffer-annotations-1.17.jar;C:\hadoop-3.3.0\share\
```

yarn resourcemanager

```
C:\Windows\system32>yarn resourcemanager
2023-11-06 11:53:32,487 INFO resourcemanager.ResourceManager: STARTUP_MSG:
/*****
STARTUP_MSG: Starting ResourceManager
STARTUP_MSG:   host = Azhar/127.0.0.1
STARTUP_MSG:   args = []
STARTUP_MSG:   version = 3.3.0
STARTUP_MSG:   classpath = C:\hadoop-3.3.0\etc\hadoop;C:\hadoop-3.3.0\etc\hadoop;C:\hadoop-3.3.0\etc\hadoop;C:\hadoop-3.3.0\share\hadoop\common;C:\hadoop-3.3.0\share\hadoop\common\lib\accessors-smart-1.2.jar;C:\hadoop-3.3.0\share\hadoop\common\lib\animal-sniffer-annotations-1.17.jar;C:\hadoop-3.3.0\share\
```

STEP 24: To check the check the daemons that are running in background we can use Java Virtual Machine Process Status which is used to list the java virtual machines that are currently running on a system it is used to display the process ID (PID) of each JVM

```
C:\Windows\System32>jps
9712 NodeManager
8212 Jps
16056 NameNode
1800 ResourceManager
18200 DataNode
```

STEP 25: Now we can access the Namenode and Datanode as web user interface (web-UI) by using the following localhost address

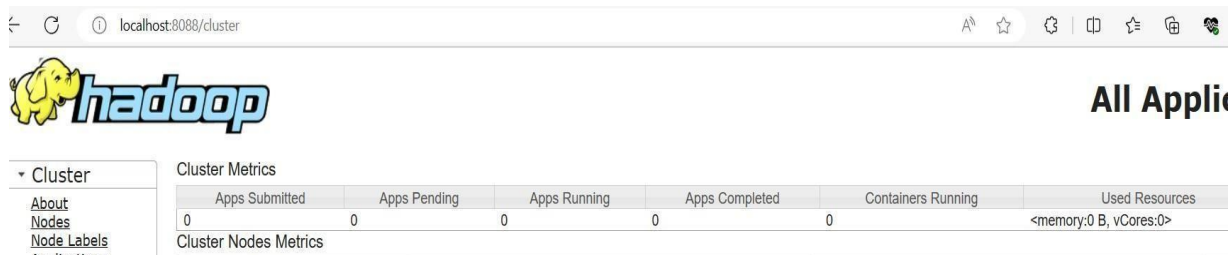
localhost:9870



Overview 'localhost:9000' (✓active)

STEP 26: To access the Datanode use the following localhost address

localhost:8088



procedure:

Procedure:

1. Install Java
→ Java is required for Hadoop to run.
2. Download Hadoop
→ Get it from the Apache website.
3. Extract Hadoop
→ Unzip the file to a folder (e.g., /usr/local/hadoop).
4. Set Environment Variables
→ Add Hadoop and Java paths in .bashrc.
5. Configure Hadoop Files
→ Edit core-site.xml, hdfs-site.xml, etc.

6. Format the NameNode

→ Run: `hdfs namenode -format`

7. Start Hadoop Services

→ Run: `start-dfs.sh` and `start-yarn.sh`

8. Check Installation

→ Use `jps` and visit <http://localhost:9870>

9. Run HDFS Commands

→ Try: `hadoop fs -mkdir /test`

RESULT: Thus, the program to install and setup Hadoop has been completed and Output is verified successfully completed.

IMPLEMENTATION OF FILE MANAGEMENT TASKS

AIM: To implement file management tasks in Hadoop file system (HDFS).

ALGORITHM:

1. Creating a directory in HDFS

SYNTAX:

```
hadoop fs -mkdir <paths>
```

EXAMPLE:

```
hadoop fs -mkdir /user
```

```
hadoop fs -mkdir /user/dir1
```

```
C:\Windows\System32>hadoop fs -mkdir /user
```

```
C:\Windows\System32>hadoop fs -mkdir /user/dir1
```

2. Listing the contents of a directory

SYNTAX:

```
hadoop fs -ls <directory name>
```

EXAMPLE:

```
hadoop fs -ls /user
```

```
hadoop fs -ls /user/dir1
```

```
C:\Windows\System32>hadoop fs -ls /user
```

```
Found 1 items
```

```
drwxr-xr-x - moham supergroup 0 2023-11-05 08:47 /user/dir1
```

```
C:\Windows\System32>hadoop fs -ls /user/dir1
```

3. Uploading and downloading a file in HDFS

SYNTAX: (UPLOAD)

```
hadoop fs -put <local file system path> <hdfs destination path>
```

EXAMPLE:

```
hadoop fs -put C:\Home\samplefile.txt.txt /user/dir1/
```

```
hadoop fs -ls /user/dir1
```

```
C:\Windows\System32>hadoop fs -put C:\Home\samplefile.txt /user/dir1
```

```
C:\Windows\System32>hadoop fs -ls /user/dir1
```

```
Found 1 items
```

```
-rw-r--r--  1 moham supergroup      13 2023-11-05 14:46 /user/dir1/samplefile.txt
```

SYNTAX: (DOWNLOAD)

```
hadoop fs -get <hdfs src> <local dst>
```

EXAMPLE:

```
hadoop fs -get /user/dir1/samplefile.txt C:\Home\Hadoopfiles
```

```
C:\Windows\System32>hadoop fs -get /user/dir1/samplefile.txt C:\Home\Hadoopfiles
```

4. See the contents of a file

SYNTAX:

```
hadoop fs -cat <path[filename]>
```

EXAMPLE:

```
hadoop fs -cat /user/dir1/samplefile.txt
```

```
C:\Windows\System32>hadoop fs -cat /user/dir1/samplefile.txt
```

```
1  
2  
3  
4  
5
```

5. Copy a file from source to destination

SYNTAX:

```
hadoop fs -cp <src> <dst>
```

EXAMPLE:

```
hadoop fs /user/dir1/samplefile.txt /user/dir2
```

```
C:\Windows\System32>hadoop fs -cp /user/dir1/samplefile.txt /user/dir2

C:\Windows\System32>hadoop fs -ls /user/dir2
Found 1 items
-rw-r--r--  1 moham supergroup      13 2023-11-05 14:57 /user/dir2/samplefile.txt
```

6. Copy a file from and to local file system to hdfs

SYNTAX: (FROM)

```
hadoop fs -copyFromLocal <local file system file path> <hdfs dst>
```

EXAMPLE:

```
hadoop fs -copyFromLocal C:\Home\test.txt /user/dir1
```

```
C:\Windows\System32>hadoop fs -copyFromLocal C:\Home\test.txt /user/dir1

C:\Windows\System32>hadoop fs -ls /user/dir1
Found 2 items
-rw-r--r--  1 moham supergroup      13 2023-11-05 14:46 /user/dir1/samplefile.txt
-rw-r--r--  1 moham supergroup       0 2023-11-05 15:00 /user/dir1/test.txt
```

SYNTAX: (TO)

```
hadoop fs -copyToLocal <hdfs src> <local dst>
```

EXAMPLE:

```
hadoop fs -copyToLocal /user/dir1/samplefile.txt C:\Home\copy
```

```
C:\Windows\System32>hadoop fs -copyToLocal /user/dir1/samplefile.txt C:\Home\copy
```

7. Move file from source to destination

SYNTAX:

```
hadoop fs -mv <src> <dst>
```

EXAMPLE:

```
hadoop fs -mv /user/dir1/test.txt /user/dir2
```

```
C:\Windows\System32>hadoop fs -ls /user/dir2
Found 2 items
-rw-r--r--  1 moham supergroup      13 2023-11-05 14:57 /user/dir2/samplefile.txt
-rw-r--r--  1 moham supergroup       0 2023-11-05 15:00 /user/dir2/test.txt
```

8. Remove a file or directory in hdfs

SYNTAX:

```
hadoop fs -rm <arg>
```

EXAMPLE:

```
hadoop fs -rm /user/dir1/samplefile.txt
```

```
C:\Windows\System32>hadoop fs -rm /user/dir1/samplefile.txt
Deleted /user/dir1/samplefile.txt
```

SYNTAX: (Recursive method for deleting directories)

```
hadoop fs -rm -r <arg>
```

EXAMPLE:

```
hadoop fs -rm -r /user/dir1
```

```
C:\Windows\System32>hadoop fs -rm -r /user/dir1
Deleted /user/dir1

C:\Windows\System32>hadoop fs -ls /user
Found 1 items
drwxr-xr-x  - moham supergroup      0 2023-11-05 15:05 /user/dir2
```

9. Display few lines of a file

SYNTAX:

```
hadoop fs -tail <path[filename]>
```

EXAMPLE:

```
hadoop fs -tail /user/dir2/samplefile.txt
```

```
C:\Windows\System32>hadoop fs -tail /user/dir2/samplefile.txt
1
2
3
4
5
```

10. Display the aggregate length of a file

SYNTAX:

```
hadoop fs -du <path>
```

EXAMPLE:

```
hadoop fs -du /user/dir2/samplefile.txt
```

```
C:\Windows\System32>hadoop fs -du /user/dir2/samplefile.txt
17 17 /user/dir2/samplefile.txt
```

procedure:

1. Start Hadoop Services

Start the HDFS daemons (NameNode, DataNode).

```
start-dfs.sh
```

2. Add Directories in HDFS

Create a new directory in HDFS.

```
hdfs dfs -mkdir /myfolder
```

3. Add (Upload) Files to HDFS

Upload a local file to the created HDFS directory.

```
hdfs dfs -put /home/user/file.txt /myfolder
```

4. Retrieve (Download) Files from HDFS

Copy a file from HDFS to the local file system.

```
hdfs dfs -get /myfolder/file.txt /home/user/
```

5. Delete Files from HDFS

Delete a specific file in HDFS.

```
hdfs dfs -rm /myfolder/file.txt
```

6. Delete Directories from HDFS

Delete the directory recursively.

```
hdfs dfs -rm -r /myfolder
```

RESULT: Thus, the implementation of file management tasks in hadoop file system was implemented and Output is verified successfully.

IMPLEMENTATION OF MATRIX MULTIPLICATION WITH HADOOP MAP REDUCE

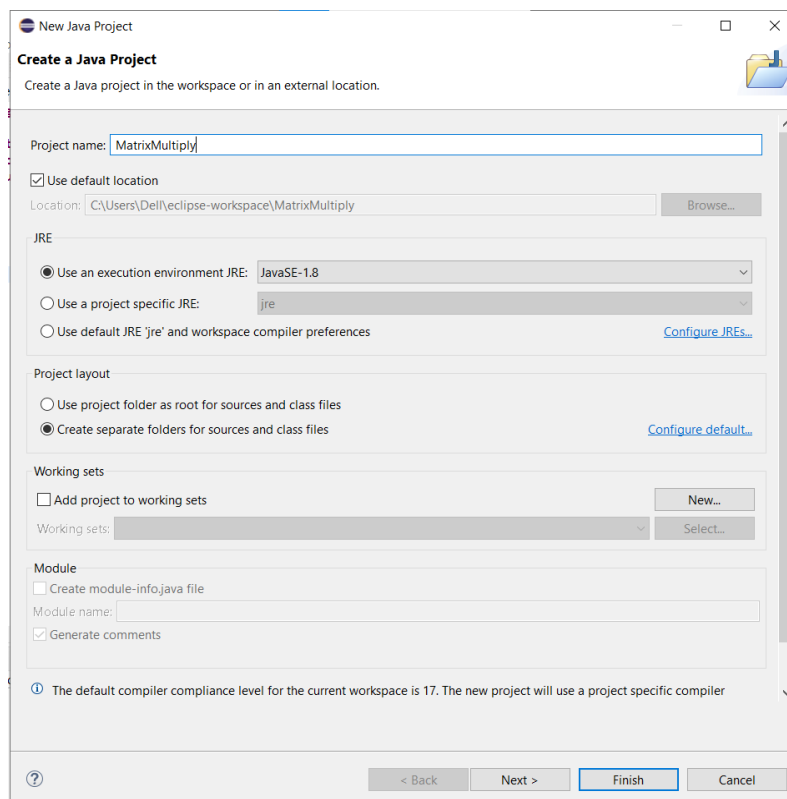
AIM: To implement of Matrix Multiplication with Hadoop Map Reduce.

ALGORITHM:

STEP 1: Run Eclipse for Java Developers

STEP 2: Create a new Java Project with name “MatrixMultiply “

STEP 3: Set the Java Environment Version to your current version of Java (JRE : 1.8)



STEP 4 : Add a Package with name “com.mapreduce.java” and Create three Classes in it.

STEP 5 : Create a New Class With name Map.java.

STEP 6: Now write the below program in the “Map.java” Class

PROGRAM:

```
package com.MapReduce.wc;

import org.apache.hadoop.conf.*;

import org.apache.hadoop.io.LongWritable;

import org.apache.hadoop.io.Text;

//import org.apache.hadoop.mapreduce.Mapper;

import java.io.IOException;

public class Map extends org.apache.hadoop.mapreduce.Mapper<LongWritable, Text, Text, Text>

{

    @Override

    public void map(LongWritable key, Text value, Context context)

    throws IOException, InterruptedException {

        Configuration conf = context.getConfiguration();

        int m = Integer.parseInt(conf.get("m"));

        int p = Integer.parseInt(conf.get("p"));

        String line = value.toString();

        // (M, i, j, Mij);

        String[] indicesAndValue = line.split(",");

        Text outputKey = new Text();

        Text outputValue = new Text();

        if (indicesAndValue[0].equals("M")) {

            for (int k = 0; k < p; k++) {
```

```

outputKey.set(indicesAndValue[1] + "," + k);

// outputKey.set(i,k);

outputValue.set(indicesAndValue[0] + "," + indicesAndValue[2]
+ "," + indicesAndValue[3]);

// outputValue.set(M,j,Mij);

context.write(outputKey, outputValue);

}

} else {

    // (N, j, k, Njk);

    for (int i = 0; i < m; i++) {

        outputKey.set(i + "," + indicesAndValue[2]); outputValue.set("N," + indicesAndValue[1] + ","
+ indicesAndValue[3]); context.write(outputKey, outputValue);

    }

}

}

```

STEP 7: Now Create another class with name “Reduce.java” and paste the below program in it.

PROGRAM:

```

package com.MapReduce.wc;

import org.apache.hadoop.io.Text;

// import org.apache.hadoop.mapreduce.Reducer;

import java.io.IOException;

import java.util.HashMap;

```

```

public class Reduce

extends org.apache.hadoop.mapreduce.Reducer<Text, Text, Text, Text> { @Override

public void reduce(Text key, Iterable<Text> values, Context context)

throws IOException, InterruptedException {

String[] value;

//key=(i,k),

//Values = [(M/N,j,V/W),..]

HashMap<Integer, Float> hashA = new HashMap<Integer, Float>(); HashMap<Integer, Float> hashB
= new HashMap<Integer, Float>(); for (Text val : values) {

value = val.toString().split(",");

if (value[0].equals("M")) {

hashA.put(Integer.parseInt(value[1]), Float.parseFloat(value[2])); } else {

hashB.put(Integer.parseInt(value[1]), Float.parseFloat(value[2]));

}

}

int n = Integer.parseInt(context.getConfiguration().get("n"));

float result = 0.0f;

float m_ij;

float n_jk;

for (int j = 0; j < n; j++) {

m_ij = hashA.containsKey(j) ? hashA.get(j) : 0.0f; n_jk = hashB.containsKey(j) ? hashB.get(j) : 0.0f;
result += m_ij * n_jk;

}

if (result != 0.0f) {

```

```

context.write(null,

new Text(key.toString() + "," + Float.toString(result)));

}

}

}

```

STEP 8: Now, Create another class with name “MatrixMultiply.java” and paste the below program in it.

PROGRAM:

```

package com.MapReduce.wc;

import org.apache.hadoop.conf.*;

import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.*;

import org.apache.hadoop.mapreduce.*;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;

import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;

import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;

public class MatrixMultiply {

    public static void main(String[] args) throws Exception { if (args.length != 2) {

        System.err.println("Usage: MatrixMultiply <in_dir> <out_dir>");

        System.exit(2);

    }

    Configuration conf = new Configuration();

```

```
conf.set("m", "1000");

conf.set("n", "100");

conf.set("p", "1000");

@SuppressWarnings("deprecation")

Job job = new Job(conf, "MatrixMultiply");

job.setJarByClass(MatrixMultiply.class);

job.setOutputKeyClass(Text.class);

job.setOutputValueClass(Text.class);

job.setMapperClass(Map.class);

job.setReducerClass(Reduce.class);

job.setInputFormatClass(TextInputFormat.class);

job.setOutputFormatClass(TextOutputFormat.class);

FileInputFormat.addInputPath(job, new Path(args[0]));

FileOutputFormat.setOutputPath(job, new Path(args[1]));

job.waitForCompletion(true);

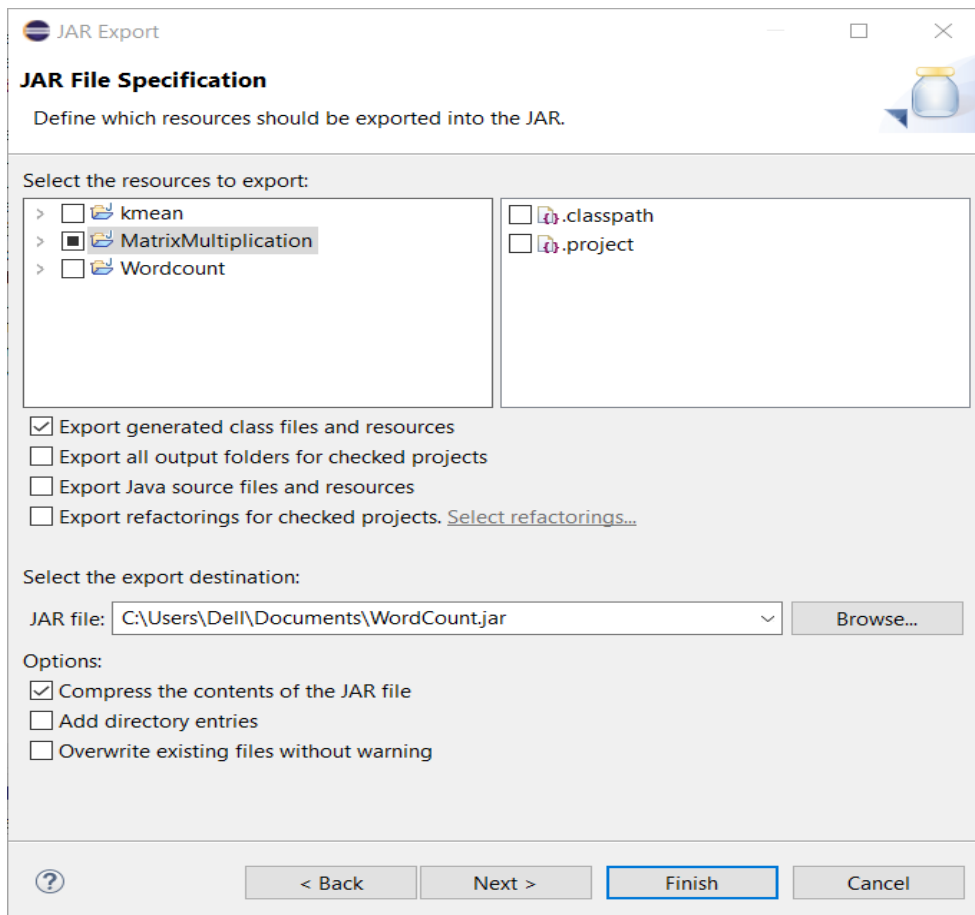
}

}
```

STEP 9: To resolve the errors in the programs we should add two External jar files to it.

- Hadoop_common :2.7.3.jar
- Hadoop_mapreduce:client:core:2.7.1.jar

STEP 10: Now export the project into a Jar file and name it as “MatrixMultiply.jar”



STEP 11: Now create a Text file in Notepad and name it as “1.txt” and “2.txt. write some content inside the text file and save it.

 1 - Notepad

File Edit Format View Help

M,0,0,1

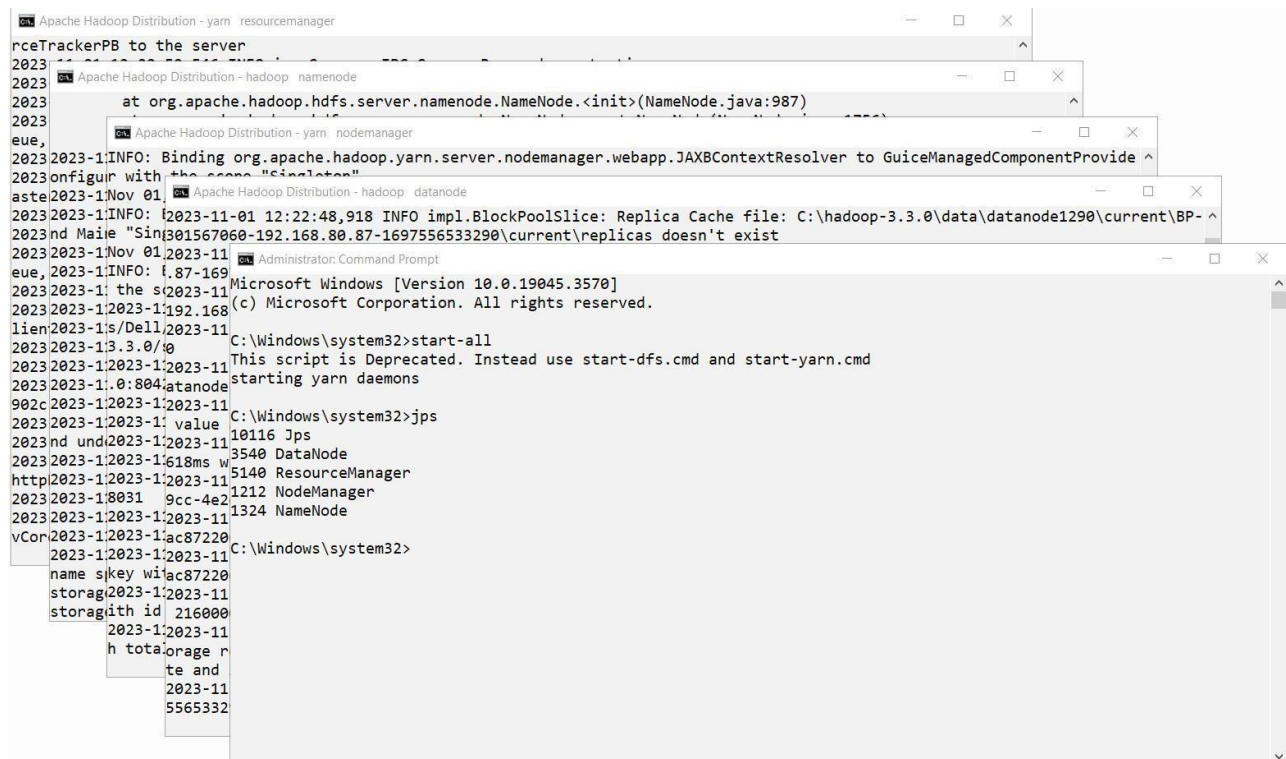
M,0,1,2

M,1,0,3

M,1,1,4

N,0,0,1
N,0,1,2
N,1,0,3
N,1,1,4

STEP 12: Now run all the daemons in Hadoop



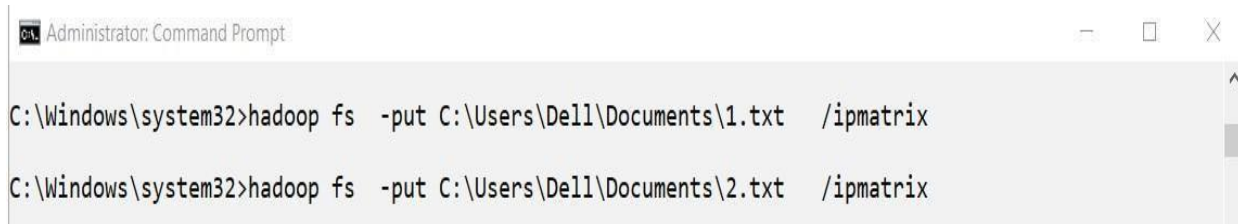
STEP 13: Create a new input directory named as “ipmatrix”.

By using the command: **hadoop fs -mkdir /ipmatrix**

STEP 14: Now put the “1.txt” and 2.txt file to the ipmatrix directory.

By using these commands: **hadoop fs -put C:\Users\Dell\Documents\1.txt /ipmatrix**

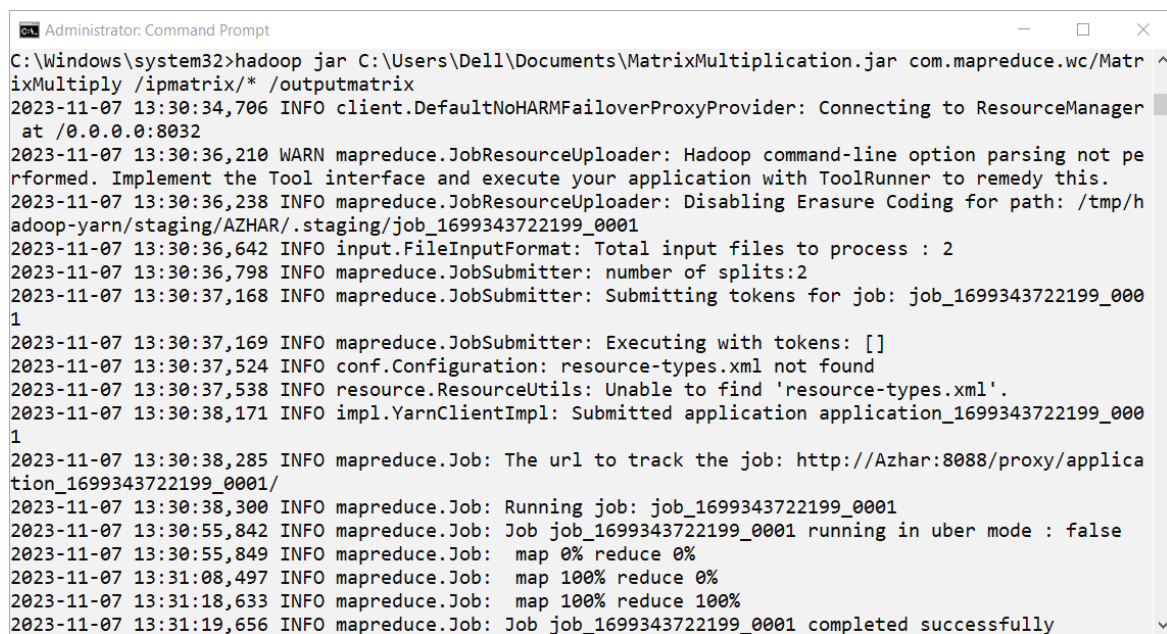
hadoop fs -put C:\Users\Dell\Documents\2.txt /ipmatrix



```
Administrator: Command Prompt
C:\Windows\system32>hadoop fs -put C:\Users\Dell\Documents\1.txt /ipmatrix
C:\Windows\system32>hadoop fs -put C:\Users\Dell\Documents\2.txt /ipmatrix
```

STEP 15: Run the Jar file created from the project

Using the command: **hadoop jar C:\Users\Dell\Documents\MatrixMultiplication.jar com.mapreduce.wc/MatrixMultiply /ipmatrix/ * /outputmatrix**



```
Administrator: Command Prompt
C:\Windows\system32>hadoop jar C:\Users\Dell\Documents\MatrixMultiplication.jar com.mapreduce.wc/MatrixMultiply /ipmatrix/* /outputmatrix
2023-11-07 13:30:34,706 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2023-11-07 13:30:36,210 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2023-11-07 13:30:36,238 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/AZHAR/.staging/job_1699343722199_0001
2023-11-07 13:30:36,642 INFO input.FileInputFormat: Total input files to process : 2
2023-11-07 13:30:36,798 INFO mapreduce.JobSubmitter: number of splits:2
2023-11-07 13:30:37,168 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1699343722199_0001
2023-11-07 13:30:37,169 INFO mapreduce.JobSubmitter: Executing with tokens: []
2023-11-07 13:30:37,524 INFO conf.Configuration: resource-types.xml not found
2023-11-07 13:30:37,538 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2023-11-07 13:30:38,171 INFO impl.YarnClientImpl: Submitted application application_1699343722199_0001
2023-11-07 13:30:38,285 INFO mapreduce.Job: The url to track the job: http://Azhar:8088/proxy/application_1699343722199_0001/
2023-11-07 13:30:38,300 INFO mapreduce.Job: Running job: job_1699343722199_0001
2023-11-07 13:30:55,842 INFO mapreduce.Job: Job job_1699343722199_0001 running in uber mode : false
2023-11-07 13:30:55,849 INFO mapreduce.Job: map 0% reduce 0%
2023-11-07 13:31:08,497 INFO mapreduce.Job: map 100% reduce 0%
2023-11-07 13:31:18,633 INFO mapreduce.Job: map 100% reduce 100%
2023-11-07 13:31:19,656 INFO mapreduce.Job: Job job_1699343722199_0001 completed successfully
```

STEP 16: At last Print your output for the MatrixMultiply text file.

Using the Command : **hadoop fs -cat /outputmatrix/***

OUTPUT :



```
Administrator: Command Prompt
C:\Windows\system32>hadoop fs -cat /outputmatrix/*
0,0,7.0
0,1,10.0
1,0,15.0
1,1,22.0
C:\Windows\system32>
```

Procedure:

Matrix Multiplication using Hadoop MapReduce - Algorithm

Let:

Matrix A of size M x N

Matrix B of size N x P

Resultant Matrix C = A x B of size M x P

Input format (in text files):

MatrixName, row_index, column_index, value

Example: A,0,1,5 or B,1,2,4

Mapper Algorithm

For each line in the input:

1. Parse line to get:

MatrixName, i, j, value

2. If MatrixName == 'A':

For k = 0 to P-1 (number of columns in B):

→ Emit key = (i,k)

→ Emit value = ('A', j, A[i][j])

3. If MatrixName == 'B':

For i = 0 to M-1 (number of rows in A):

→ Emit key = (i,k)

→ Emit value = ('B', j, B[j][k])

Reducer Algorithm

For each key (i,k):

1. Create two lists:

listA = [] for all A[i][j]

listB = [] for all B[j][k]

2. Group and store values:

For each value:

If source is 'A', add to listA

If source is 'B', add to listB

3. Compute dot product:

For each element in listA:

For each element in listB:

If index j matches → multiply $A[i][j] \times B[j][k]$

Add to sum

4. Emit:

→ Key: (i,k)

→ Value: sum

RESULT : Thus the program to run a basic wordcount mapreduce program to understand mapreduce is executed and output is verified successfully.

RUN A BASIC WORD COUNT MAP REDUCE PROGRAM TO UNDERSTAND MAP REDUCE PARADIGM

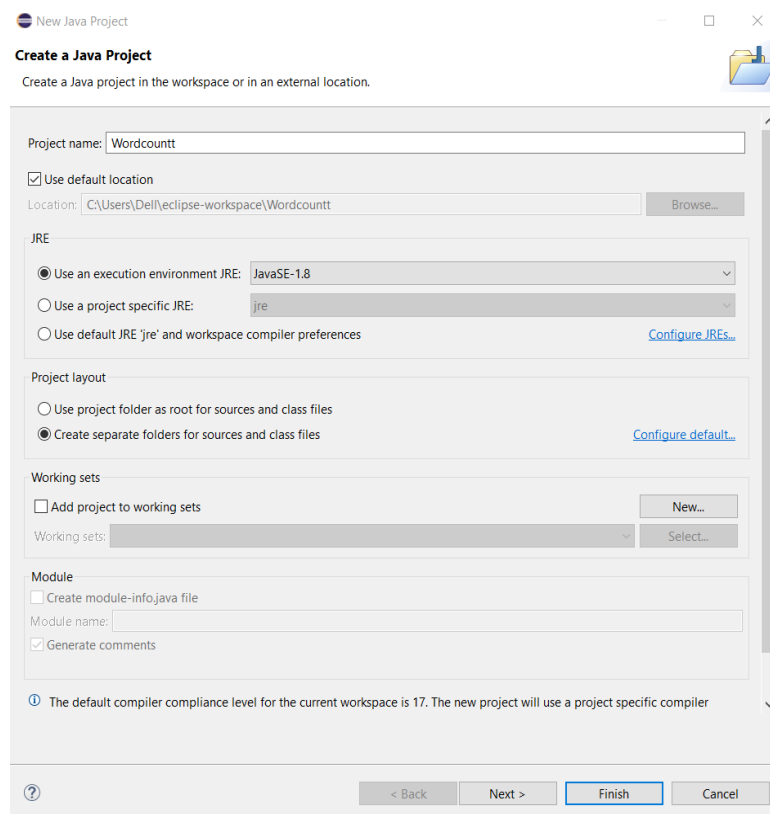
AIM: To Run a Basic Word Count Map Reduce program to understand Map Reduce Paradigm.

ALGORITHM:

STEP 1: Run Eclipse for Java Developers

STEP 2: Create a new Java Project with name “WordCount “

STEP 3: Set the Java Environment Version to your current version of Java (JRE : 1.8)



STEP 4 : Add a Package with name “com.mapreduce.java” and Create three Classes in it.

STEP 5 : Create a New Class With name WC_Mapper.java.

STEP 6: Now write the below program in the “WC_Mapper.java” Class

PROGRAM:

```
package com.mapreduce.java;

import java.io.IOException;

import java.util.StringTokenizer;

import org.apache.hadoop.io.IntWritable;

import org.apache.hadoop.io.LongWritable;

import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapred.MapReduceBase;

import org.apache.hadoop.mapred.Mapper;

import org.apache.hadoop.mapred.OutputCollector;

import org.apache.hadoop.mapred.Reporter;

public class WC_Mapper extends MapReduceBase implements
Mapper<LongWritable,Text,Text,IntWritable>{

    private final static IntWritable one = new IntWritable(1);

    private Text word = new Text();

    public void map(LongWritable key, Text value,OutputCollector<Text,IntWritable> output,

        Reporter reporter) throws IOException{

        String line = value.toString();

        StringTokenizer tokenizer = new StringTokenizer(line);

        while (tokenizer.hasMoreTokens()){

            word.set(tokenizer.nextToken());

            output.collect(word, one);

        } }

}
```

STEP 7: Now Create another class with name “WC_Reducer.java” and paste the below program in it.

PROGRAM:

```
package com.mapreduce.java;

import java.io.IOException;

import java.util.Iterator;

import org.apache.hadoop.io.IntWritable;

import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapred.MapReduceBase;

import org.apache.hadoop.mapred.OutputCollector;

import org.apache.hadoop.mapred.Reducer;

import org.apache.hadoop.mapred.Reporter;

public class WC_Reducer extends MapReduceBase implements
Reducer<Text,IntWritable,Text,IntWritable> {

public void reduce(Text key, Iterator<IntWritable> values,OutputCollector<Text,IntWritable> output,

Reporter reporter) throws IOException {

int sum=0;

while (values.hasNext()) {

sum+=values.next().get();

}

output.collect(key,new IntWritable(sum));

}

}
```

STEP 8: Now, Create another class with name “WC_runner.java” and paste the below program in it.

PROGRAM:

```
package com.mapreduce.java;

import java.io.IOException;

import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;

import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapred.FileInputFormat;

import org.apache.hadoop.mapred.FileOutputFormat;

import org.apache.hadoop.mapred.JobClient;

import org.apache.hadoop.mapred.JobConf;

import org.apache.hadoop.mapred.TextInputFormat;

import org.apache.hadoop.mapred.TextOutputFormat;

public class WC_Runner {

    public static void main(String[] args) throws IOException{

        JobConf conf = new JobConf(WC_Runner.class);

        conf.setJobName("WordCount");

        conf.setOutputKeyClass(Text.class);

        conf.setOutputValueClass(IntWritable.class);

        conf.setMapperClass(WC_Mapper.class);

        conf.setCombinerClass(WC_Reducer.class);

        conf.setReducerClass(WC_Reducer.class);

        conf.setInputFormat(TextInputFormat.class);
```

```

conf.setOutputFormat(TextOutputFormat.class);

FileInputFormat.setInputPaths(conf,new Path(args[0]));

FileOutputFormat.setOutputPath(conf,new Path(args[1]));

JobClient.runJob(conf);

}

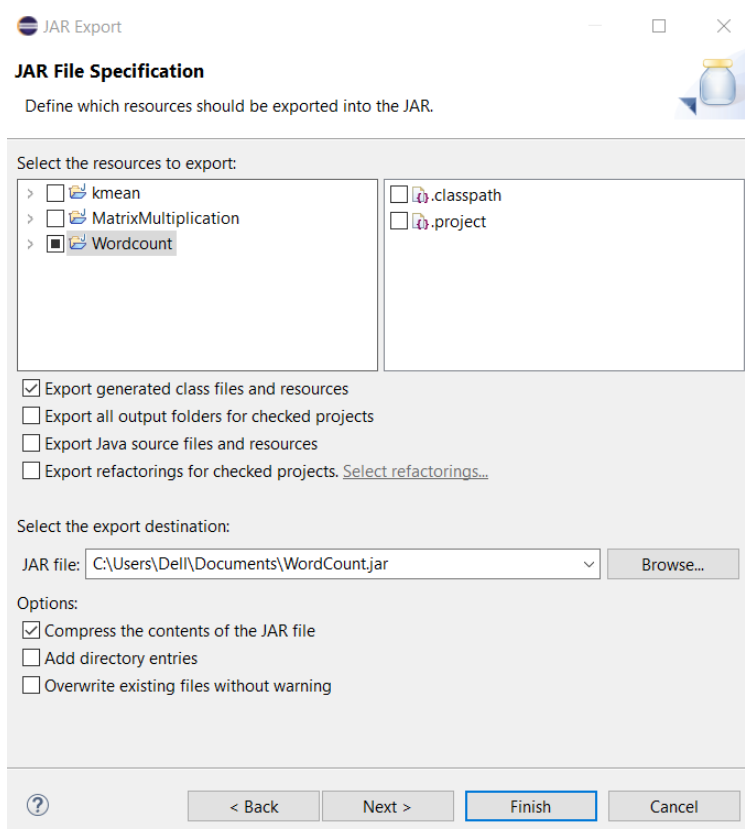
}

```

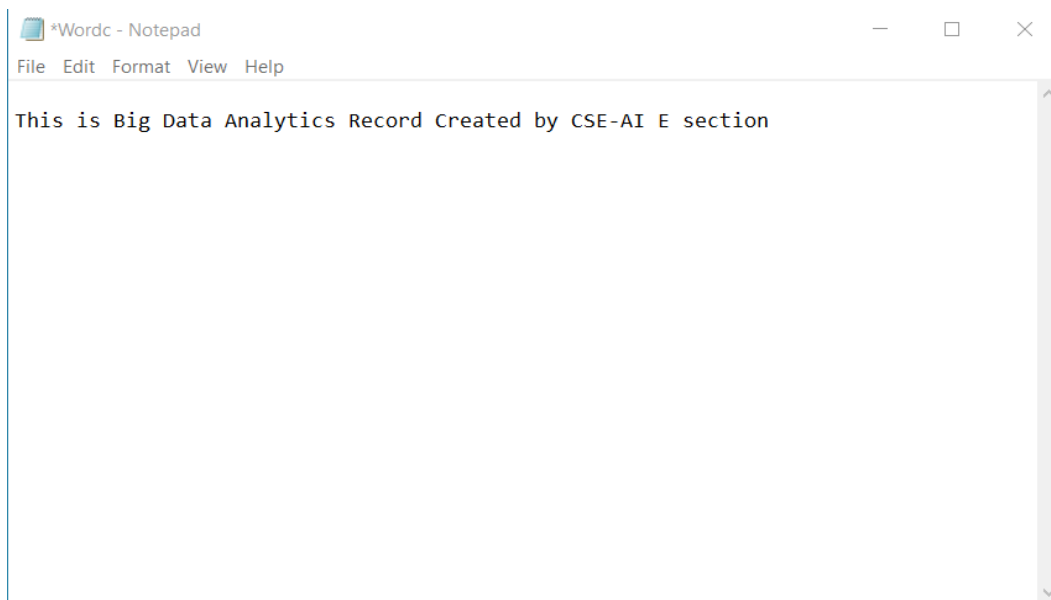
STEP 9: To resolve the errors in the programs we should add two External jar files to it.

- Hadoop_common :2.7.3.jar
- Hadoop_mapreduce:client:core:2.7.1.jar

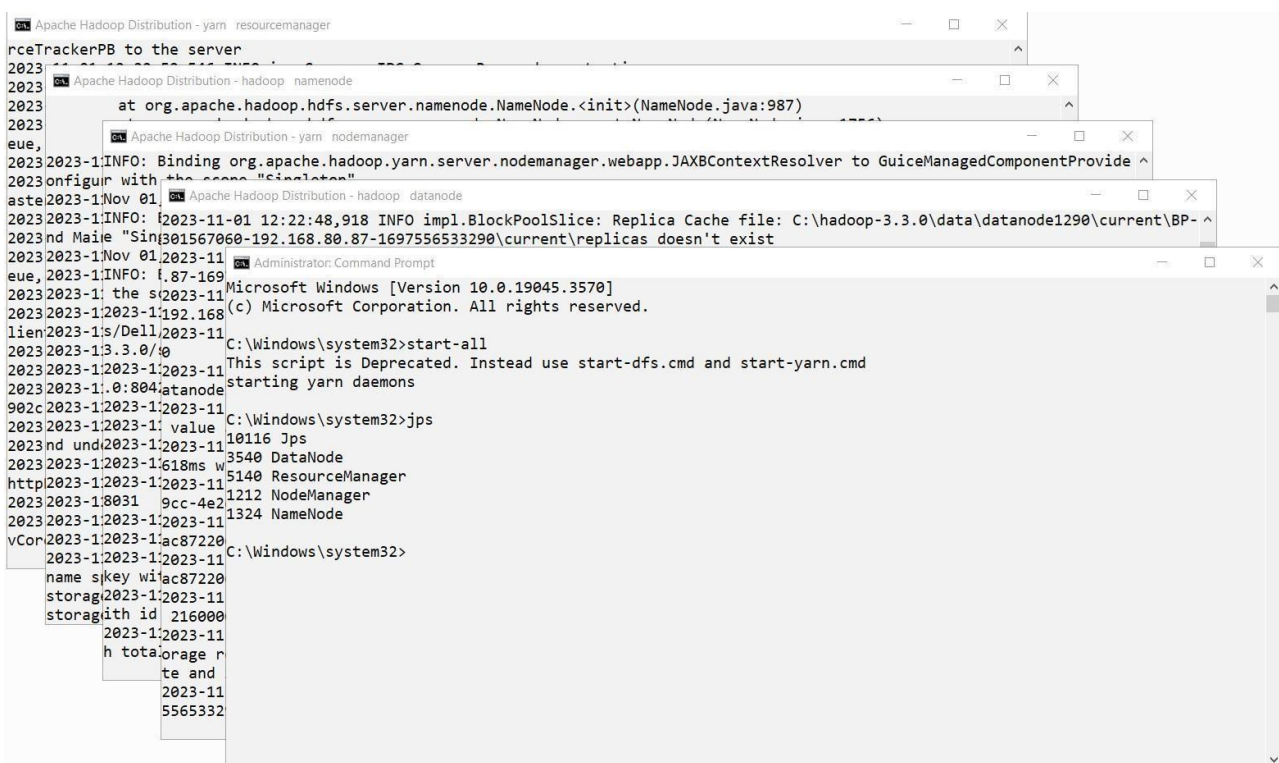
STEP 10: Now export the project into a Jar file and name it as “WordCount.jar”



STEP 11: Now create a Text file in Notepad and name it as “wordc.txt” and write some content inside the text file and save it.



STEP 12: Now run all the demons in Hadoop.



STEP 13: Create a new input directory named as “inputword”.

By using the command: `hadoop fs -mkdir /inputword`

STEP 14: Now put the “wordc.txt” file to the inputword directory.

By using the command: `hadoop fs -put C:\Users\Dell\Documents\wordc.txt /inputword`

```
Administrator: Command Prompt
C:\hadoop-3.3.0\sbin>hadoop fs -mkdir /inputword

C:\hadoop-3.3.0\sbin>hadoop fs -put C:\Users\Dell\Documents\wordc.txt /inputword

C:\hadoop-3.3.0\sbin>hadoop jar C:\Users\Dell\Documents\Wordcount.jar com.mapreduce.java/WC_Runner /inputword/*
/outputword
2023-11-01 12:30:22,811 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /0.0.0
.0:8032
2023-11-01 12:30:23,141 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /0.0.0
.0:8032
2023-11-01 12:30:23,890 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. I
mplement the Tool interface and execute your application with ToolRunner to remedy this.
2023-11-01 12:30:23,910 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn
/staging/AZHAR/.staging/job_1698821571744_0001
2023-11-01 12:30:24,368 INFO mapred.FileInputFormat: Total input files to process : 1
2023-11-01 12:30:24,512 INFO mapreduce.JobSubmitter: number of splits:2
2023-11-01 12:30:24,841 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1698821571744_0001
2023-11-01 12:30:24,845 INFO mapreduce.JobSubmitter: Executing with tokens: []
2023-11-01 12:30:25,265 INFO conf.Configuration: resource-types.xml not found
2023-11-01 12:30:25,265 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2023-11-01 12:30:25,930 INFO impl.YarnClientImpl: Submitted application application_1698821571744_0001
2023-11-01 12:30:26,004 INFO mapreduce.Job: The url to track the job: http://Azhar:8088/proxy/application_16988
21571744_0001/
2023-11-01 12:30:26,008 INFO mapreduce.Job: Running job: job_1698821571744_0001
2023-11-01 12:30:40,397 INFO mapreduce.Job: Job job_1698821571744_0001 running in uber mode : false
```

STEP 15: Run the Jar file created from the project

Using the command: `hadoop jar C:\Users\Dell\Documents\Wordcount.jar com.mapreduce.java/WC_Runner /inputword/* /outputword`

```
Administrator: Command Prompt
C:\hadoop-3.3.0\sbin>hadoop jar C:\Users\Dell\Documents\Wordcount.jar com.mapreduce.java/WC_Runner /inputword/*
/outputword
2023-11-01 12:30:22,811 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /0.0.0
.0:8032
2023-11-01 12:30:23,141 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /0.0.0
.0:8032
2023-11-01 12:30:23,890 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. I
mplement the Tool interface and execute your application with ToolRunner to remedy this.
2023-11-01 12:30:23,910 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn
/staging/AZHAR/.staging/job_1698821571744_0001
2023-11-01 12:30:24,368 INFO mapred.FileInputFormat: Total input files to process : 1
2023-11-01 12:30:24,512 INFO mapreduce.JobSubmitter: number of splits:2
2023-11-01 12:30:24,841 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1698821571744_0001
2023-11-01 12:30:24,845 INFO mapreduce.JobSubmitter: Executing with tokens: []
2023-11-01 12:30:25,265 INFO conf.Configuration: resource-types.xml not found
2023-11-01 12:30:25,265 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2023-11-01 12:30:25,930 INFO impl.YarnClientImpl: Submitted application application_1698821571744_0001
2023-11-01 12:30:26,004 INFO mapreduce.Job: The url to track the job: http://Azhar:8088/proxy/application_16988
21571744_0001/
2023-11-01 12:30:26,008 INFO mapreduce.Job: Running job: job_1698821571744_0001
2023-11-01 12:30:40,397 INFO mapreduce.Job: Job job_1698821571744_0001 running in uber mode : false
2023-11-01 12:30:40,399 INFO mapreduce.Job: map 0% reduce 0%
2023-11-01 12:30:51,769 INFO mapreduce.Job: map 100% reduce 0%
2023-11-01 12:31:00,913 INFO mapreduce.Job: map 100% reduce 100%
2023-11-01 12:31:01,959 INFO mapreduce.Job: Job job_1698821571744_0001 completed successfully
2023-11-01 12:31:02,167 INFO mapreduce.Job: Counters: 54
```

STEP 16: At last Print your output for the WordCount text file.

Using the Command : `hadoop fs -cat /outputword/*`

OUTPUT:



```
Administrator: Command Prompt
C:\hadoop-3.3.0\sbin>hadoop fs -cat /outputword/*
Analytics      1
Big            1
CSE-AI         1
Created        1
Data           1
E              1
Record         1
This           1
by             1
is             1
section        1

C:\hadoop-3.3.0\sbin>
```

Procedure:

1. Input: A collection of text documents.

2. Map Phase:

For each document:

Split the document into individual words (tokens).

For each word:

Output a key-value pair: (word, 1)

Example: For the document "the cat sat on the mat", the mapper would output:

("the", 1)

("cat", 1)

("sat", 1)

("on", 1)

("the", 1)

("mat", 1)

3. Shuffle and Sort (Implicit in MapReduce framework):

The framework automatically shuffles and sorts the output of the map phase based on the keys (words). This ensures that all occurrences of the same word are grouped together.

4. Reduce Phase:

For each unique word (key):

Sum all the associated values (which are all 1s in this case) to get the total count for that word.

Example: For the word "the", the reducer would sum the "1"s, resulting in the output ("the", 2)

Output the word and its total count.

RESULT: Thus the program to run a basic wordcount mapreduce program to understand mapreduce is executed and output is verified successfully.

IMPLEMENTATION OF K-MEANS CLUSTERING USING MAPREDUCE

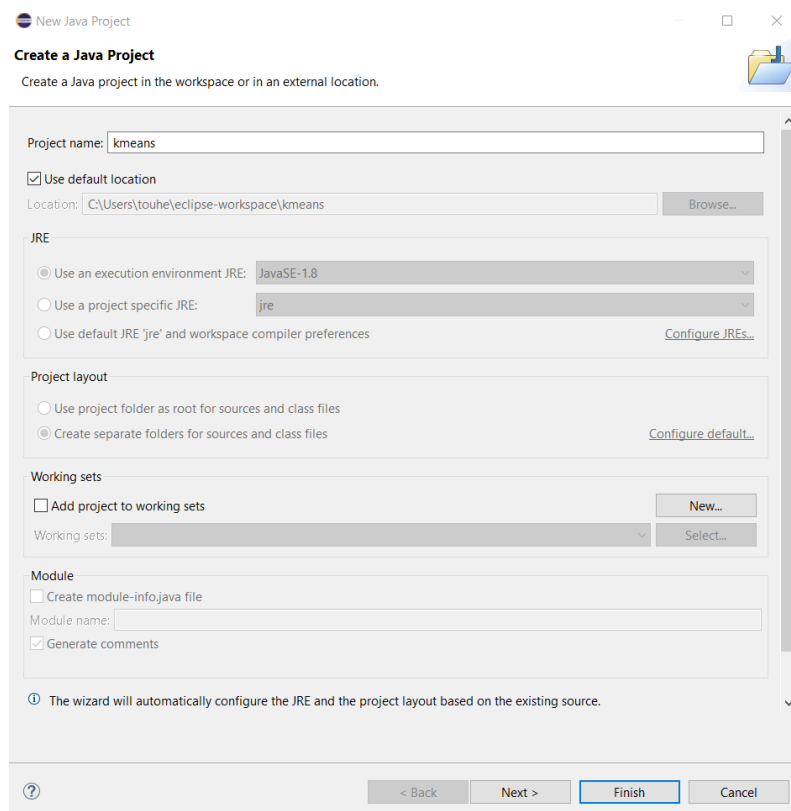
AIM : To implement K-means clustering using mapreduce.

ALGORITHM:

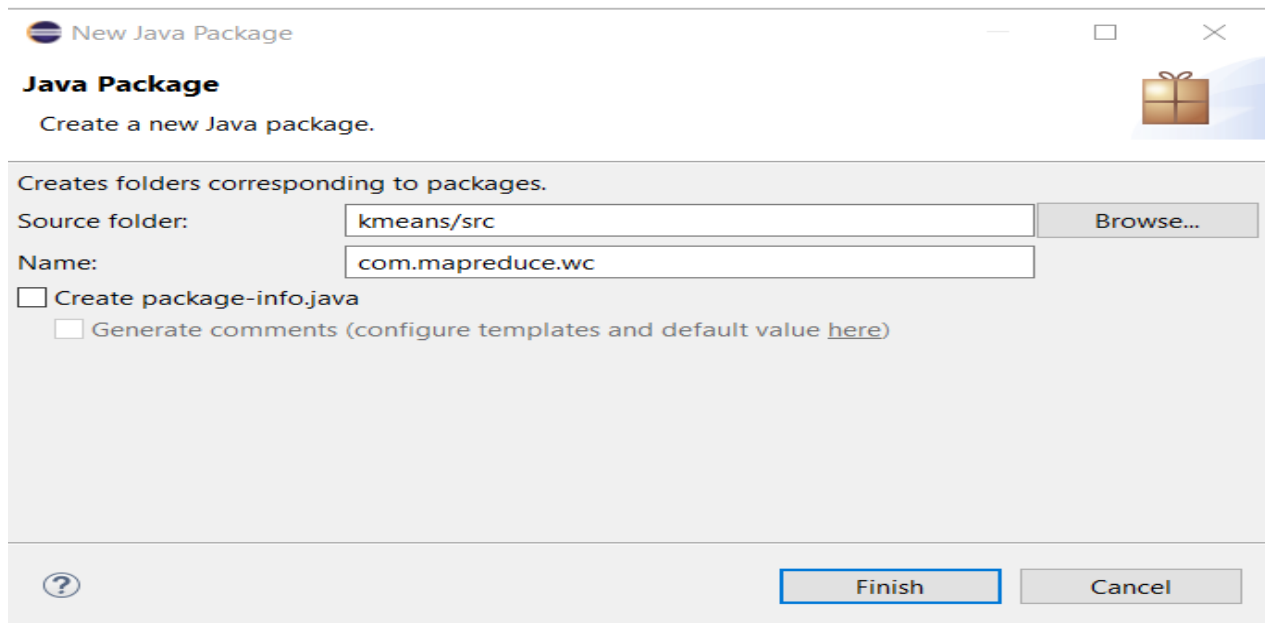
STEP 1: Run Eclipse for Java Developers

STEP 2: Create a new Java Project with name “Kmeans “.

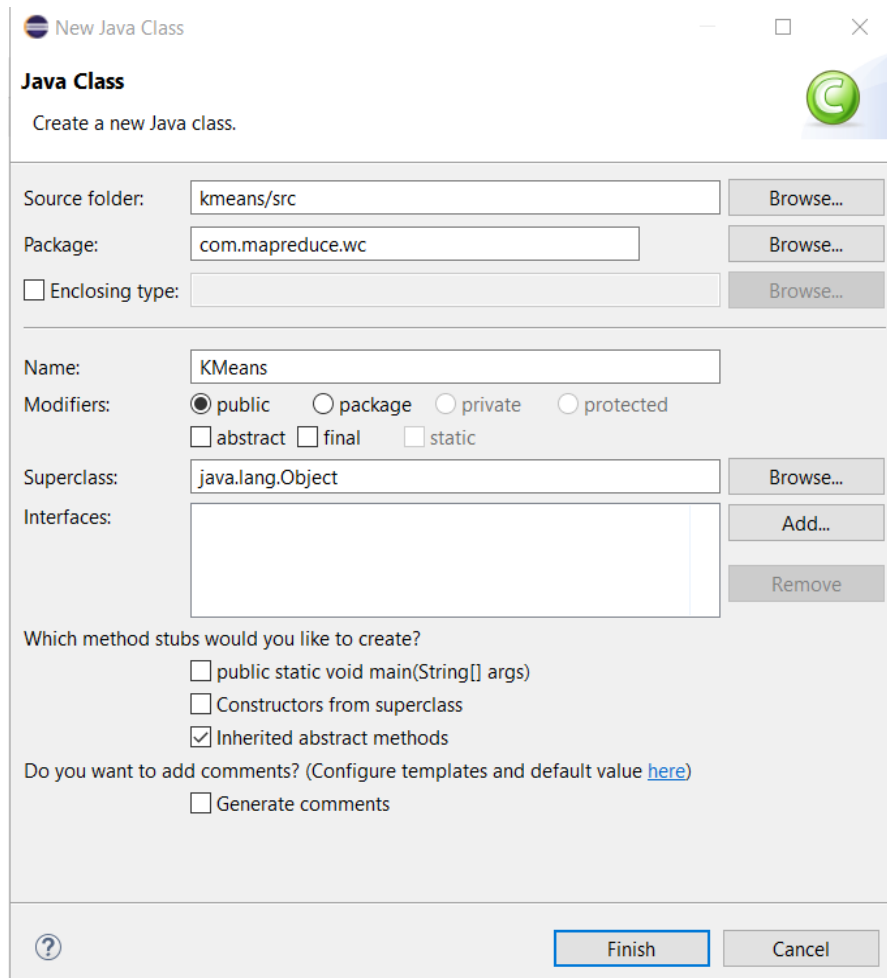
STEP 3: Set the Java Environment Version to your current version of Java (JRE - 1.8)



STEP 4: Add a Package with name “com.mapreduce.java” and Create three Classes in it.



STEP 5 :Create a New Class With name “KMeans.java”



STEP 6: Now write the below program in the “KMeans.java” Class

PROGRAM :

```
package com.mapreduce.wc;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.Iterator;
import java.util.List;
import java.util.Map;

public class KMeans {

    List<Record> data = new ArrayList<Record>();
    List<Cluster> clusters = new ArrayList<Cluster>();
    Map<Cluster, List<Record>> clusterRecords = new HashMap<Cluster, List<Record>>();

    public static void main(String[] args) {
        int clusterNumber = 2;
        KMeans demo = new KMeans();
        demo.genereateRecord();
        demo.initiateClusterAndCentroid(clusterNumber);
        demo.printRecordInformation();
        demo.printClusterInformation();
    }

    private void genereateRecord() {
        Record record = new Record(1, 19, 15, 39);
        data.add(record);
        record = new Record(2, 21, 15, 81);
        data.add(record);

        record = new Record(3, 20, 16, 6);
        data.add(record);
        record = new Record(4, 23, 16, 77);
        data.add(record);
        record = new Record(5, 31, 17, 40);
        data.add(record);
        record = new Record(6, 22, 17, 76);
        data.add(record);
    }

    private void initiateClusterAndCentroid(int clusterNumber) {
```

```

int counter = 1;
Iterator<Record> iterator = data.iterator();
Record record = null;
while(iterator.hasNext()) {
    record = iterator.next();
    if(counter <= clusterNumber) {
        record.setClusterNumber(counter);
        initializeCluster(counter, record);
        counter++;
    } else {
        System.out.println(record);
        System.out.println("*** Cluster Information ***");
        for(Cluster cluster : clusters) {
            System.out.println(cluster);
        }
        System.out.println("*****");
    }
}

double minDistance = Integer.MAX_VALUE;
Cluster whichCluster = null;

for(Cluster cluster : clusters) {
    double distance = cluster.calculateDistance(record);
    System.out.println(distance);
    if(minDistance > distance) {
        minDistance = distance;
        whichCluster = cluster;
    }
}

record.setClusterNumber(whichCluster.getClusterNumber());
whichCluster.updateCentroid(record);
clusterRecords.get(whichCluster).add(record);
}
System.out.println("*** Cluster Information ***");

for(Cluster cluster : clusters) {
    System.out.println(cluster);
}

```

```

        }
        System.out.println("*****");
    }
}

private void initializeCluster(int clusterNumber, Record record) {Cluster
    System.out.println("***** Each Record INFORMATIN *****");
    for(Record record : data) {
        System.out.println(record);
    }clusterCluster(clusterNumber,record.getAge(),record.getIncome(),record.getScore());
    clusters.add(cluster);
    List<Record> clusterRecord = new ArrayList<Record>();
    clusterRecord.add(record);
    clusterRecords.put(cluster, clusterRecord);
}

private void printRecordInformation() {
}

private void printClusterInformation() {
    System.out.println("***** FINAL CLUSTER INFORMATIN *****");
    for (Map.Entry<Cluster, List<Record>> entry : clusterRecords.entrySet()) {
System.out.println("Key = " + entry.getKey() +
        ", Value = " + entry.getValue());
    }
}
}
}

```

STEP 7: Now Create another class with name “Cluster.java” and write the below program in it.

PROGRAM :

```
package com.mapreduce.wc;

public class Cluster {
    private int ageCentroid;
    private int incomeCentroid;
    private int scoreCentroid;
    private int clusterNumber;

    public Cluster(int clusterNumber, int ageCentroid, int incomeCentroid, int scoreCentroid) {
        super();
        this.clusterNumber = clusterNumber;
        this.ageCentroid = ageCentroid;
        this.incomeCentroid = incomeCentroid;
        this.scoreCentroid = scoreCentroid;
    }

    public int getAgeCentroid() {
        return ageCentroid;
    }

    public void setAgeCentroid(int ageCentroid) {
        this.ageCentroid = ageCentroid;
    }

    public int getIncomeCentroid() {
        return incomeCentroid;
    }

    public void setIncomeCentroid(int incomeCentroid) {
        this.incomeCentroid = incomeCentroid;
    }

    public int getScoreCentroid() {
        return scoreCentroid;
    }

    public void setScoreCentroid(int scoreCentroid) {
        this.scoreCentroid = scoreCentroid;
    }

    public int getClusterNumber() {
        return clusterNumber;
    }
}
```

```

    }

    public void setClusterNumber(int clusterNumber) {
        this.clusterNumber = clusterNumber;
    }

    @Override
    public String toString() {
        return "Cluster [ageCentroid=" + ageCentroid + ", incomeCentroid=" + incomeCentroid + ",
scoreCentroid="
                + scoreCentroid + ", clusterNumber=" + clusterNumber + "]";
    }

    // Euclidean distance calculation
    public double calculateDistance(Record record) {
        return Math.sqrt(Math.pow((getAgeCentroid() - record.getAge()), 2) +
Math.pow((getIncomeCentroid() - record.getIncome()),2) + Math.pow((getScoreCentroid() -
record.getScore()), 2));
    }

    // Binod Suman Academy YouTube Video on K-Mean Algorithm
    public void updateCentroid(Record record) {
        setAgeCentroid((getAgeCentroid()+record.getAge())/2);
        setIncomeCentroid((getIncomeCentroid()+record.getIncome())/2);
        setScoreCentroid((getScoreCentroid()+record.getScore())/2);
    }
}

```

STEP 8: Now Create another class with name “Record.java” and write the below program in it.

PROGRAM :

```

package com.mapreduce.wc;

public class Record {
    private int id;
    private int age;
    private int income;
    private int score;
    private int clusterNumber

    public Record(int id, int age, int income, int score) {
        super();
    }
}

```

```
        this.id = id;
        this.age = age;
        this.income = income;
        this.score = score;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }
    public int getIncome() {
        return income;
    }
    public void setIncome(int income) {
        this.income = income;
    }
    public int getScore() {
        return score;
    }
    public void setScore(int score) {
        this.score = score;
    }
    public int getClusterNumber() {
        return clusterNumber;
    }
    public void setClusterNumber(int clusterNumber) {
        this.clusterNumber = clusterNumber;
    }
}
```

```

}

@Override

public String toString() {

    return "Record [id=" + id + ", age=" + age + ", income=" + income + ", score=" + score + ",

    clusterNumber=" + clusterNumber + "]\n";

}

}

```

STEP 9: Run the “KMeans.java” class from the project to get the output.

OUTPUT:

```

eclipse-workspace - kmeans/src/com/mapreduce/wc/KMeans.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
-terminated- KMeans [Java Application] C:\Users\touhelp\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64.17.0.8.v20230831-1047\jre\bin\javaw.exe (04-Oct-2023, 12:47:49 pm - 12:47:49 pm) [pid: 6636]

** Cluster Information **
Cluster [ageCentroid=19, incomeCentroid=15, scoreCentroid=39, clusterNumber=1]
Cluster [ageCentroid=21, incomeCentroid=15, scoreCentroid=81, clusterNumber=2]
*****
Record [id=3, age=20, income=16, score=6, clusterNumber=0]
** Cluster Information **
Cluster [ageCentroid=19, incomeCentroid=15, scoreCentroid=39, clusterNumber=1]
Cluster [ageCentroid=21, incomeCentroid=15, scoreCentroid=81, clusterNumber=2]
*****
33.03028912982749
75.0133321483588
** Cluster Information **
Cluster [ageCentroid=19, incomeCentroid=15, scoreCentroid=22, clusterNumber=1]
Cluster [ageCentroid=21, incomeCentroid=15, scoreCentroid=81, clusterNumber=2]
*****
Record [id=4, age=23, income=16, score=77, clusterNumber=0]
** Cluster Information **
Cluster [ageCentroid=19, incomeCentroid=15, scoreCentroid=22, clusterNumber=1]
Cluster [ageCentroid=21, incomeCentroid=15, scoreCentroid=81, clusterNumber=2]
*****
55.154328932550705
4.5825756945594
** Cluster Information **
Cluster [ageCentroid=19, incomeCentroid=15, scoreCentroid=22, clusterNumber=1]
Cluster [ageCentroid=22, incomeCentroid=15, scoreCentroid=79, clusterNumber=2]
*****
Record [id=5, age=31, income=17, score=40, clusterNumber=0]
** Cluster Information **
Cluster [ageCentroid=19, incomeCentroid=15, scoreCentroid=22, clusterNumber=1]
Cluster [ageCentroid=22, incomeCentroid=15, scoreCentroid=79, clusterNumber=2]
*****
21.72550698240943
40.07492981902776
** Cluster Information **
Cluster [ageCentroid=25, incomeCentroid=16, scoreCentroid=31, clusterNumber=1]
Cluster [ageCentroid=22, incomeCentroid=15, scoreCentroid=79, clusterNumber=2]
*****
Record [id=6, age=22, income=17, score=76, clusterNumber=0]
** Cluster Information **

```

```
eclipse-workspace - kmeans/src/com/mapreduce/wc/KMeans.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

<terminated> KMeans [Java Application] C:\Users\touhe\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.17.0.8\j20230831-1047\jre\bin\javaw.exe (04-Oct-2023, 12:47:49 pm - 12:47:49 pm) [pid: 6636]

Cluster [ageCentroid=19, incomeCentroid=15, scoreCentroid=22, clusterNumber=1]
Cluster [ageCentroid=21, incomeCentroid=15, scoreCentroid=81, clusterNumber=2]
*****
55.154328932550705
4.58257569495584
** Cluster Information **
Cluster [ageCentroid=19, incomeCentroid=15, scoreCentroid=22, clusterNumber=1]
Cluster [ageCentroid=22, incomeCentroid=15, scoreCentroid=79, clusterNumber=2]
*****
Record [id=5, age=31, income=17, score=40, clusterNumber=0]
** Cluster Information **
Cluster [ageCentroid=19, incomeCentroid=15, scoreCentroid=22, clusterNumber=1]
Cluster [ageCentroid=22, incomeCentroid=15, scoreCentroid=79, clusterNumber=2]
*****
21.72556098240043
40.07492981902776
** Cluster Information **
Cluster [ageCentroid=25, incomeCentroid=16, scoreCentroid=31, clusterNumber=1]
Cluster [ageCentroid=22, incomeCentroid=15, scoreCentroid=79, clusterNumber=2]
*****
Record [id=6, age=22, income=17, score=76, clusterNumber=0]
** Cluster Information **
Cluster [ageCentroid=25, incomeCentroid=16, scoreCentroid=31, clusterNumber=1]
Cluster [ageCentroid=22, incomeCentroid=15, scoreCentroid=79, clusterNumber=2]
*****
45.110974274559844
3.605551275463989
** Cluster Information **
Cluster [ageCentroid=25, incomeCentroid=16, scoreCentroid=31, clusterNumber=1]
Cluster [ageCentroid=22, incomeCentroid=16, scoreCentroid=77, clusterNumber=2]
*****
***** Each Record INFORMATION *****
Record [id=1, age=19, income=15, score=39, clusterNumber=1]
Record [id=2, age=21, income=15, score=81, clusterNumber=2]
Record [id=3, age=20, income=16, score=6, clusterNumber=1]
Record [id=4, age=23, income=16, score=77, clusterNumber=2]
Record [id=5, age=31, income=17, score=40, clusterNumber=1]
Record [id=6, age=22, income=17, score=76, clusterNumber=2]
***** FINAL CLUSTER INFORMATION *****
Key = Cluster [ageCentroid=25, incomeCentroid=16, scoreCentroid=31, clusterNumber=1], Value = [Record [id=1, age=19, income=15, score=39, clusterNumber=1], Record [id=3, age=20, income=16, score=6, clusterNumber=1]]
Key = Cluster [ageCentroid=22, incomeCentroid=16, scoreCentroid=77, clusterNumber=2], Value = [Record [id=2, age=21, income=15, score=81, clusterNumber=2], Record [id=4, age=23, income=16, score=77, clusterNumber=2], Record [id=5, age=31, income=17, score=40, clusterNumber=1], Record [id=6, age=22, income=17, score=76, clusterNumber=2]]
```

Procedure:

Step 1: Initialization

Choose K initial centroids randomly.

Save them in a file (e.g., centroids.txt).

Mapper Algorithm

For each data point (x, y):

1. Load current centroids
2. Compute distance of point to each centroid
3. Find the nearest centroid
4. Emit key = centroidID, value = dataPoint

Reducer Algorithm

For each centroidID:

1. Collect all assigned data points
2. Compute new centroid:

new_x = avg of x

$\text{new_y} = \text{avg of } y$

3. Emit updated centroid

Step 2: Iterate

Replace old centroids with new centroids

Repeat MapReduce job until:

Centroids do not change, or

Max iterations reached

RESULT: Thus the program to implement K-means clustering using mapreduce is executed the output is verified successfully.

INSTALLATION OF HIVE WITH PRACTICE EXAMPLES

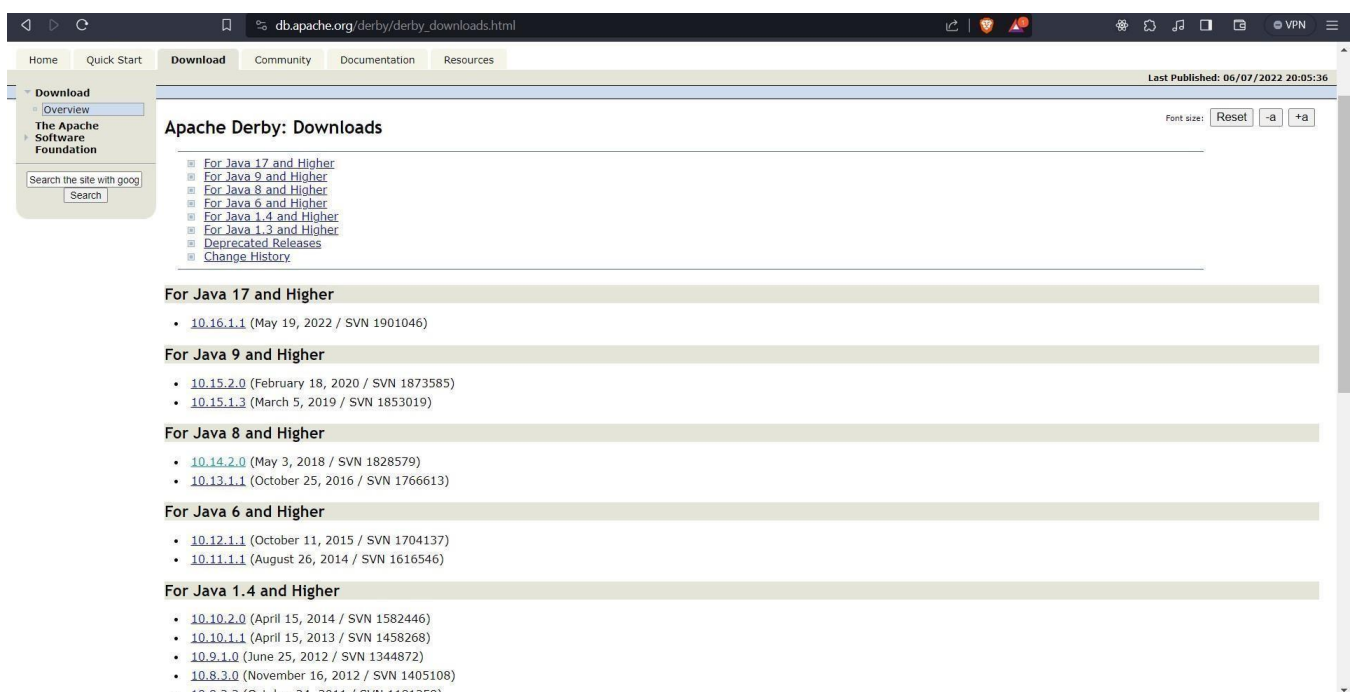
AIM: To install Hive on Windows operating system with practice examples.

ALGORITHM:

STEP 1: Install Apache Derby from the official derby website

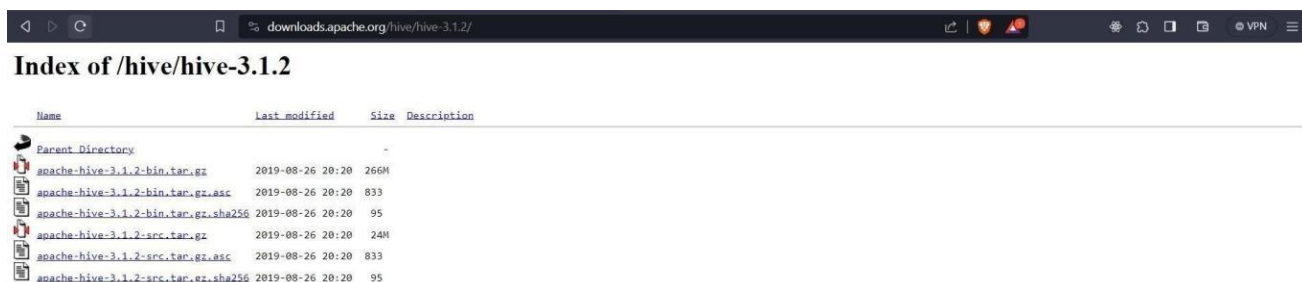
https://db.apache.org/derby/derby_downloads.html according to your Java Version

Installed on your system.

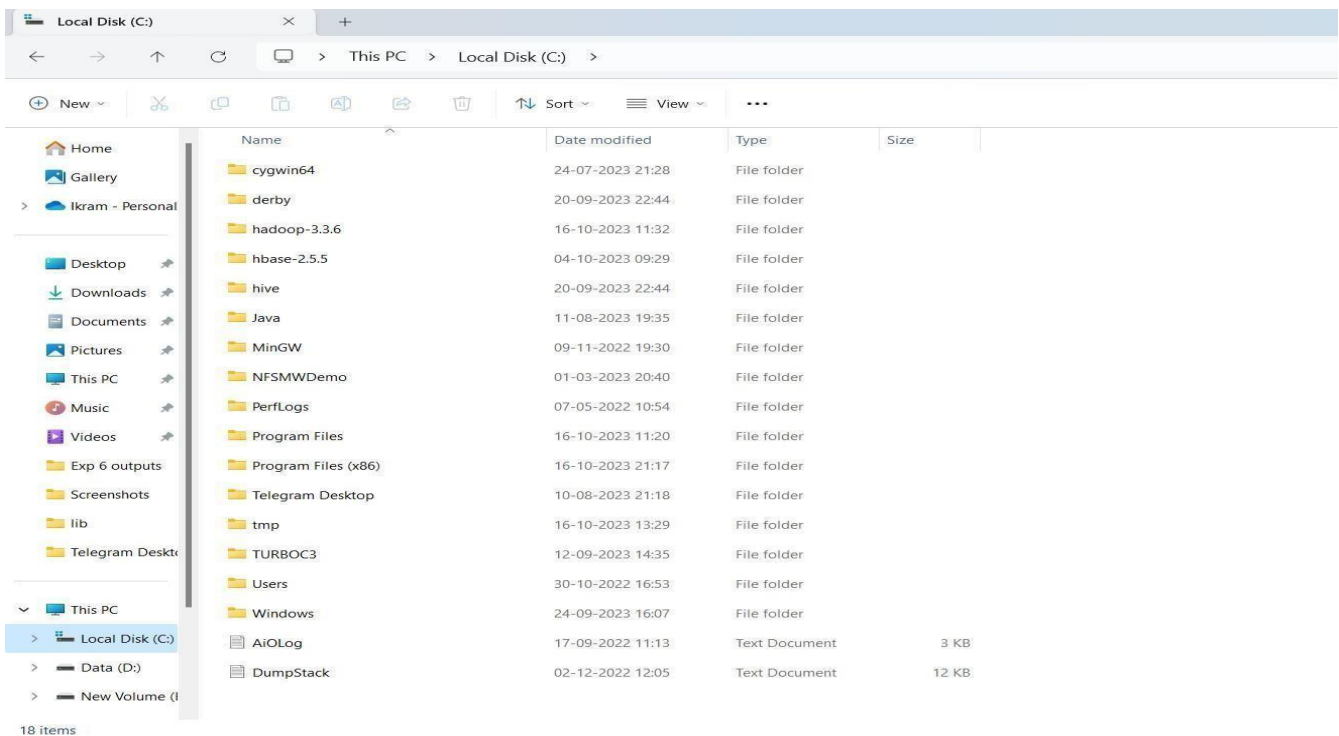


STEP 2: Install Apache Hive 3.1.2 version from the official website

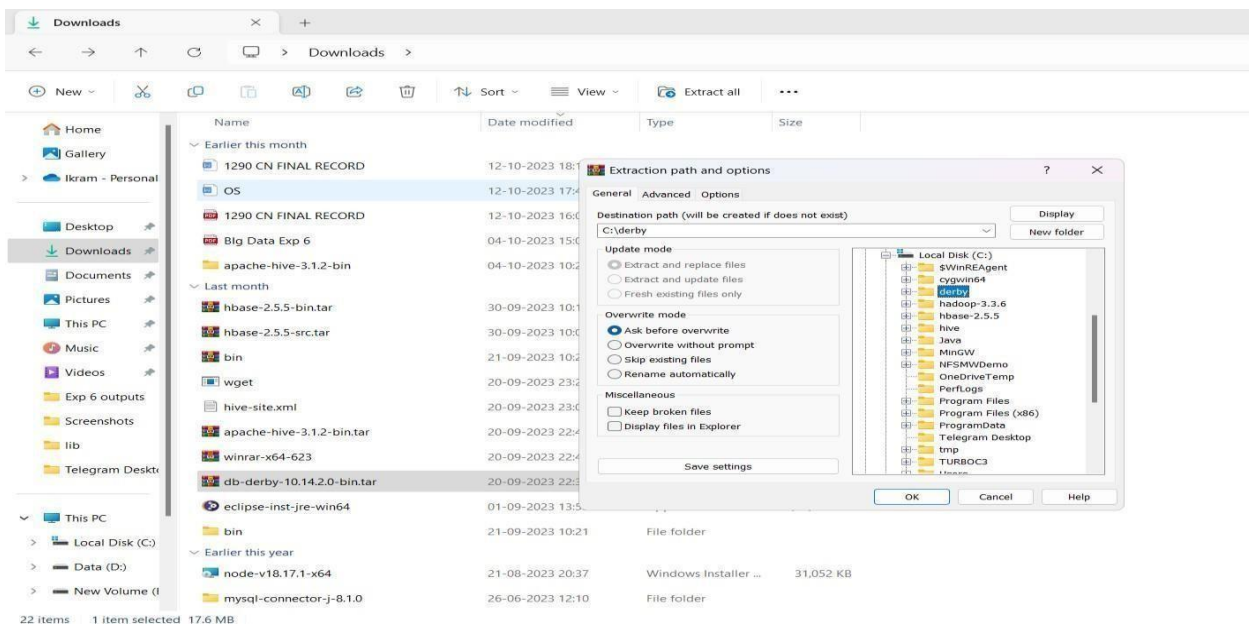
<https://downloads.apache.org/hive/hive-3.1.2/>



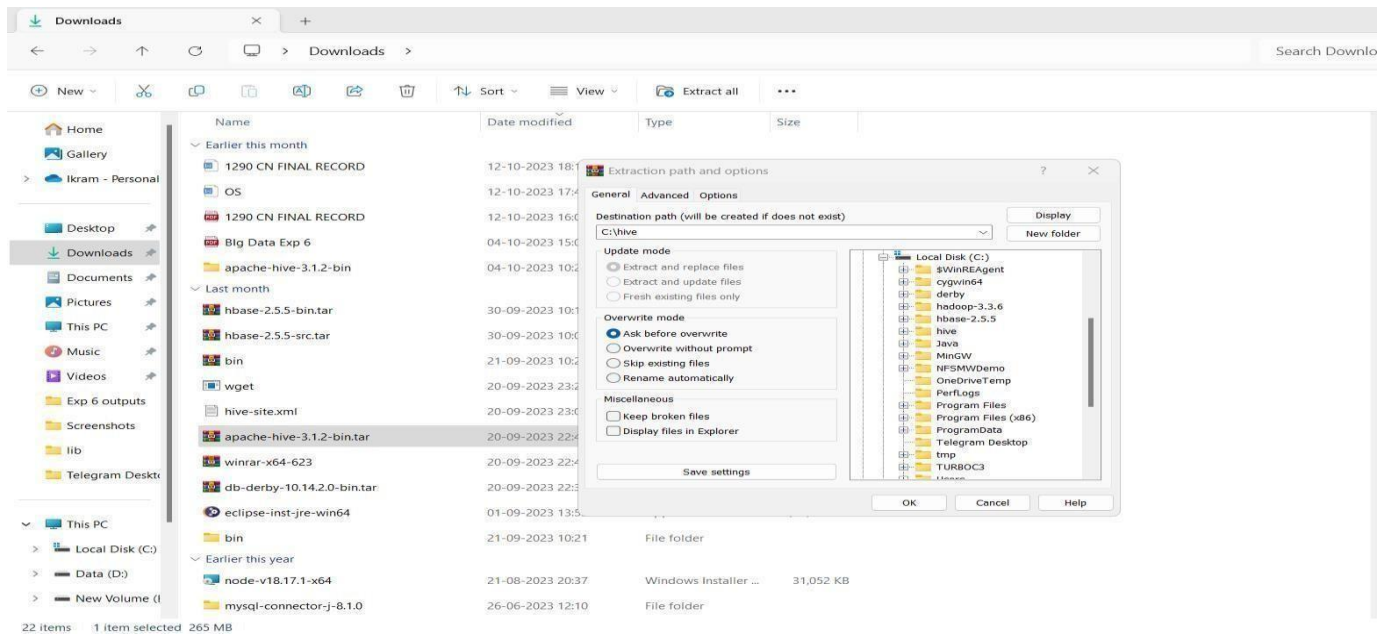
STEP 3: Create two new folders with name **Hive** and **Derby** in **C:** folder



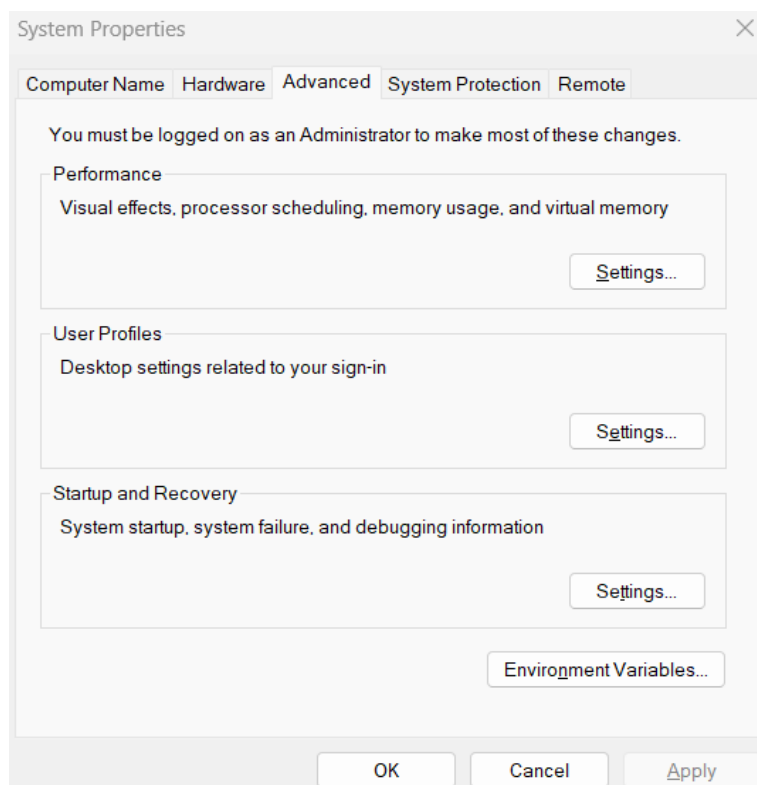
STEP 4: Extract the downloaded **Derby** file to newly created **Derby** folder in **C:** directory



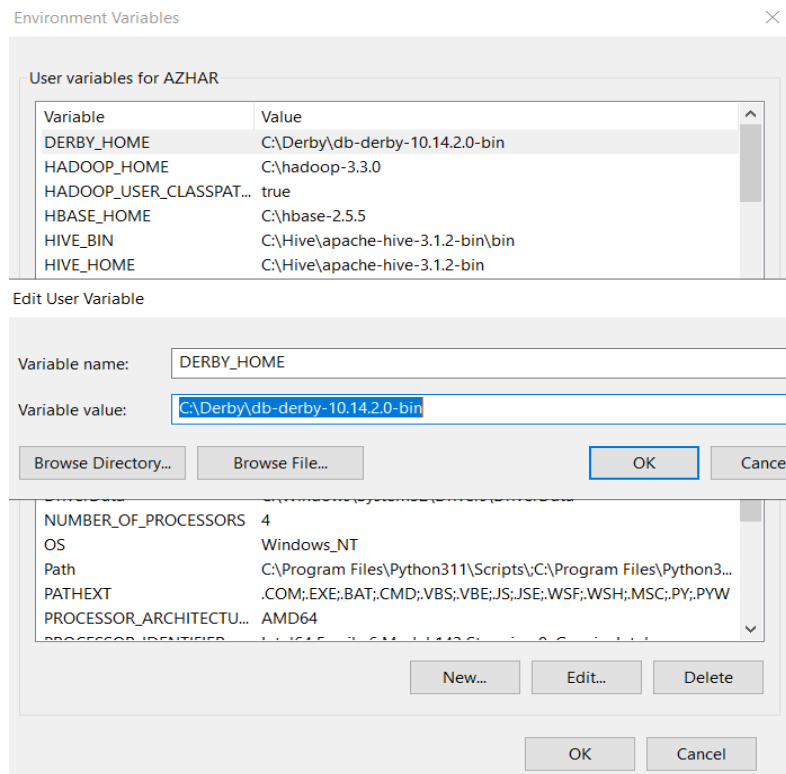
STEP 5: Extract the downloaded **Hive** file to newly created **Hive** folder in **C:** directory.



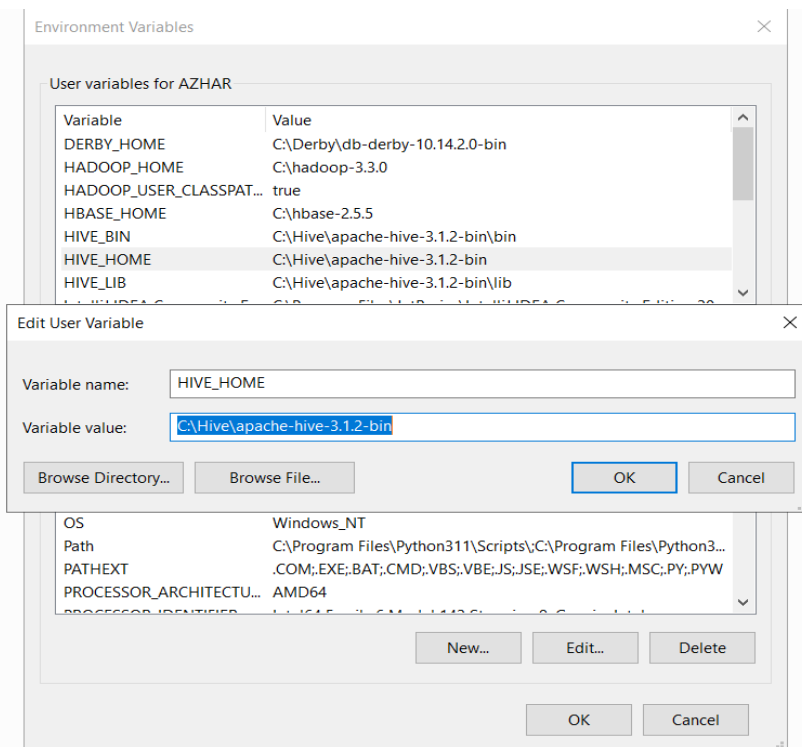
STEP 6: Now edit the Environment Variables.



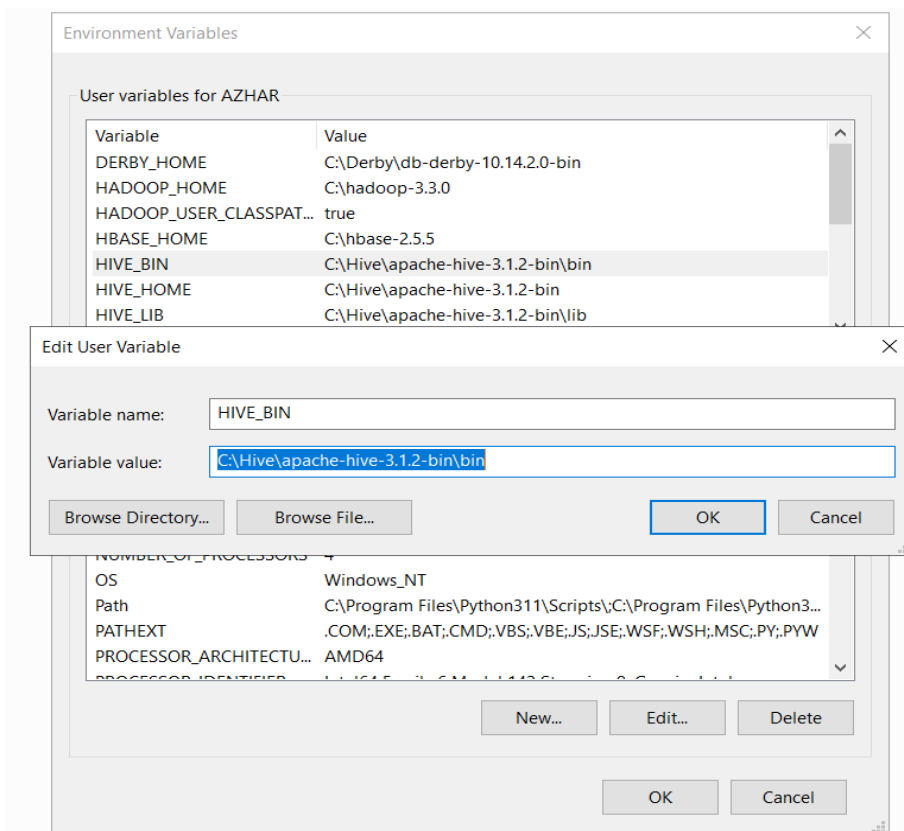
STEP 7: Create a new User Variable with Variable Name: DERBY_HOME
and with **Variable Value: C:\derby\db-derby-10.14.2.0-bin**



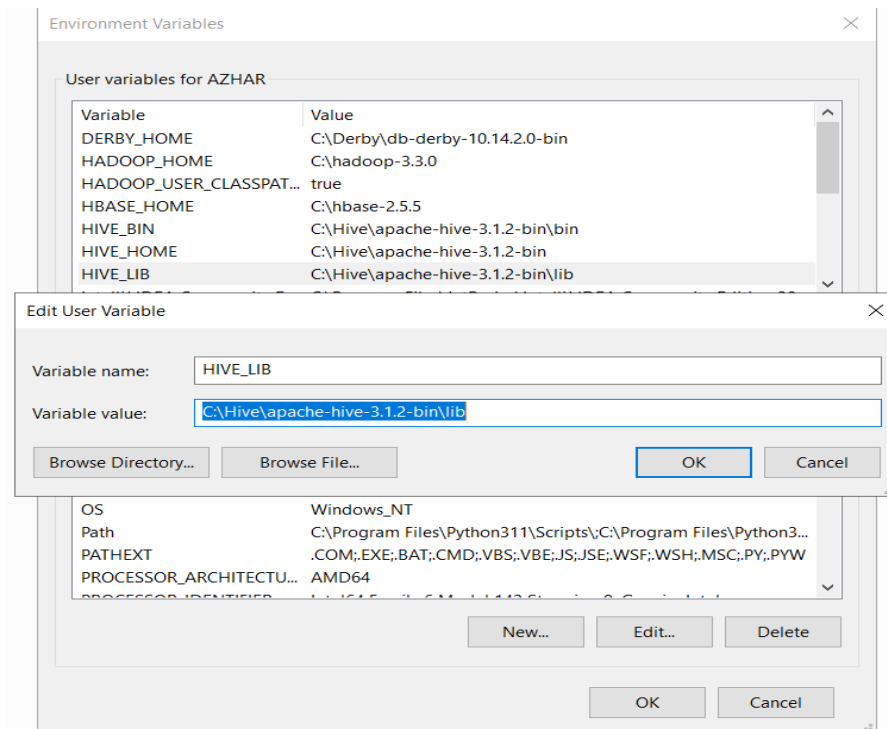
STEP 8: Create a new User Variable with Variable Name: HIVE_HOME
and with **Variable Value: C:\hive\apache-hive-3.1.2-bin**



STEP 9: Create a new User Variable with **Variable Name: HIVE_BIN** and with **Variable Value: C:\hive\apache-hive-3.1.2-bin\bin**



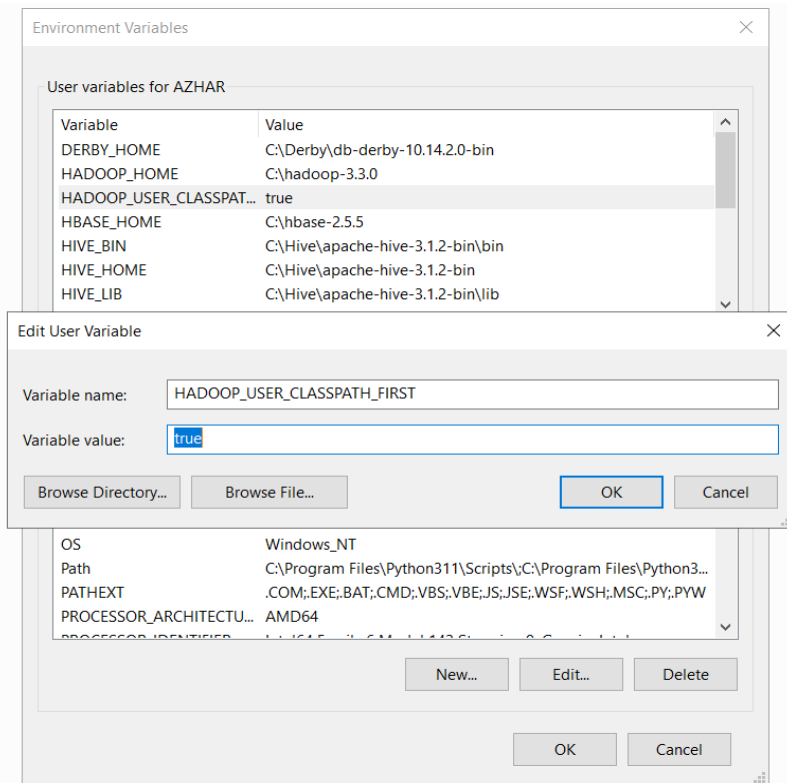
STEP 10: Create a new User Variable with **Variable Name: HIVE_LIB** and with **Variable Value: C:\hive\apache-hive-3.1.2-bin\lib**



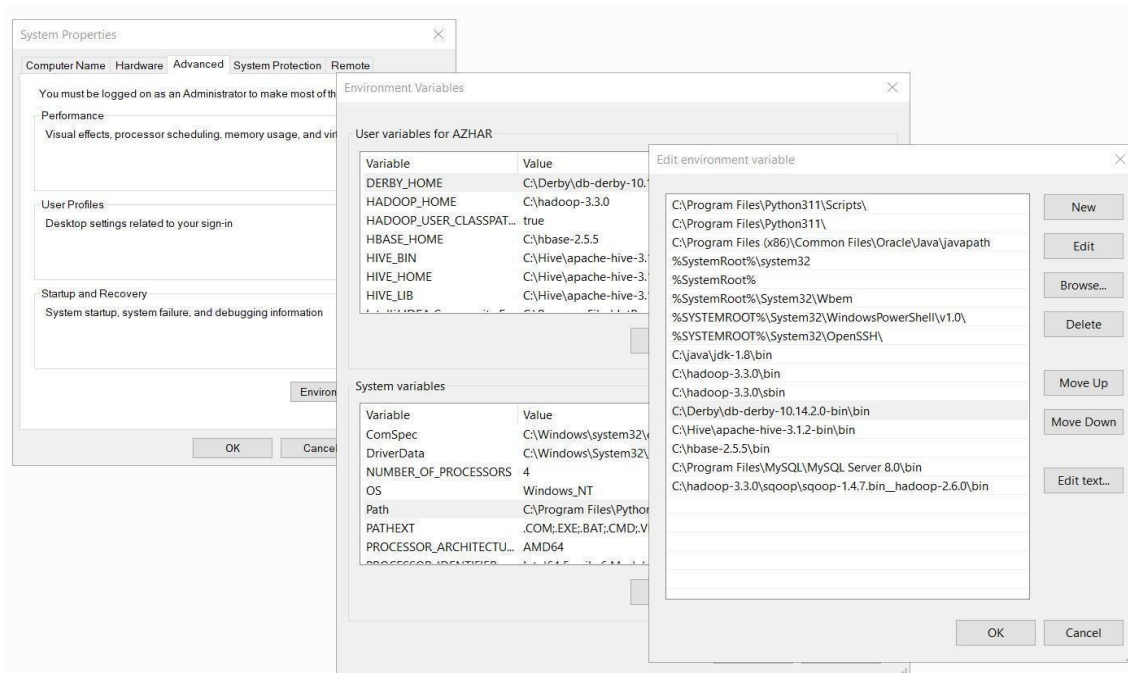
STEP 11: Create a new User Variable with

Variable Name: HADOOP_USER_CLASSPATH_FIRST and with

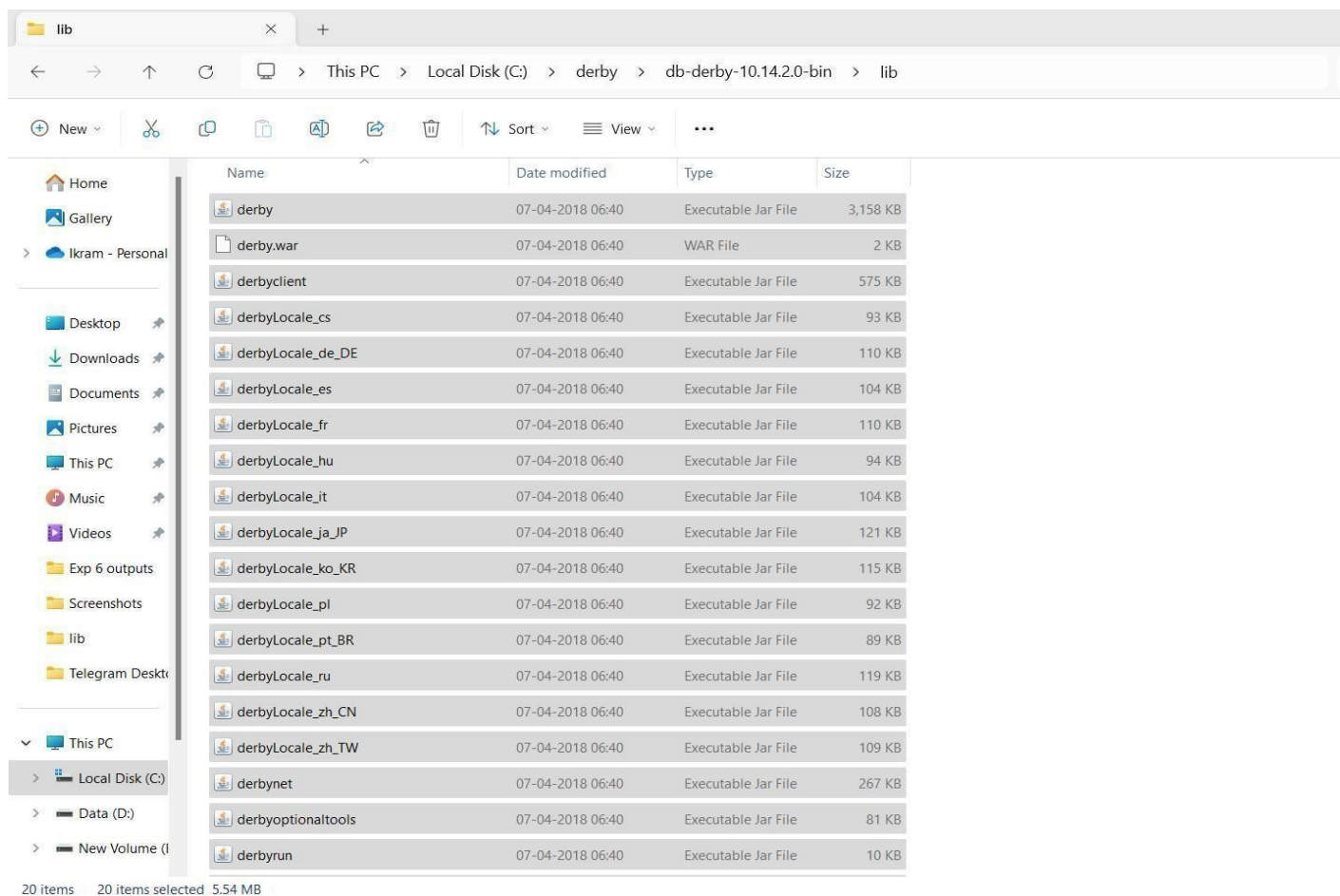
Variable Value: true



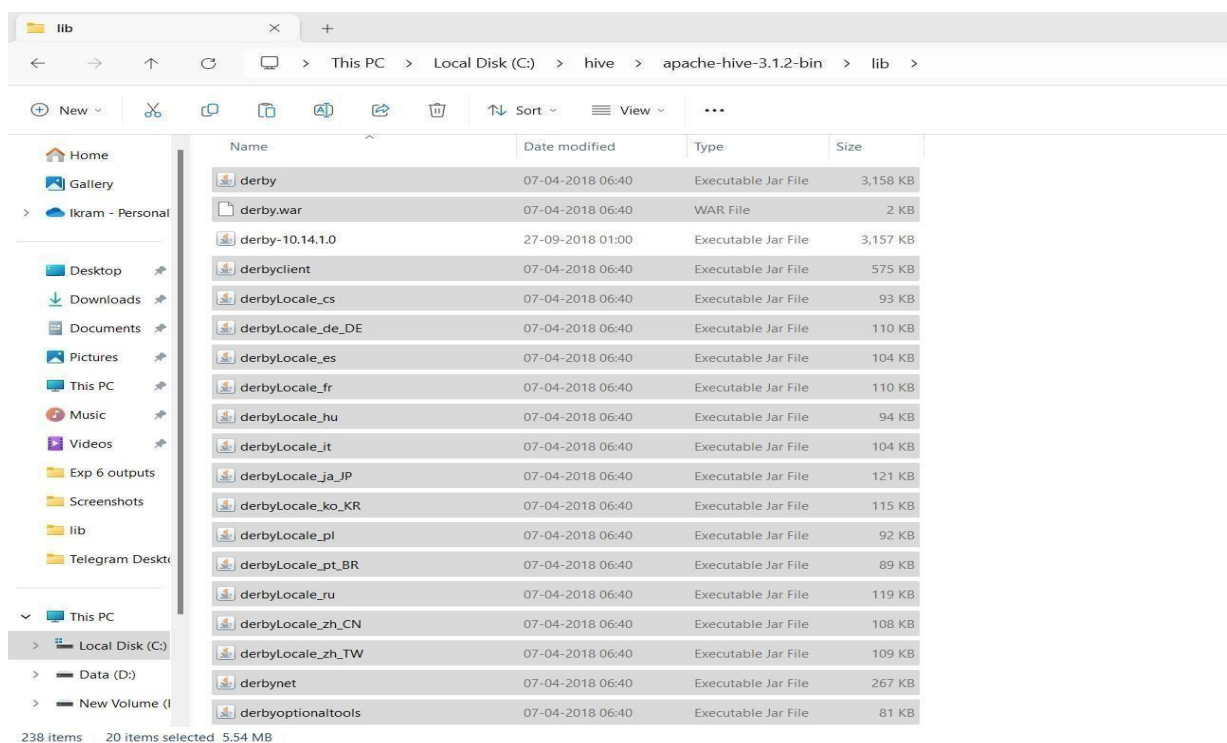
STEP 12: Add %HIVE_BIN% and %DERBY_HOME%\bin to the Path of the System Variables.



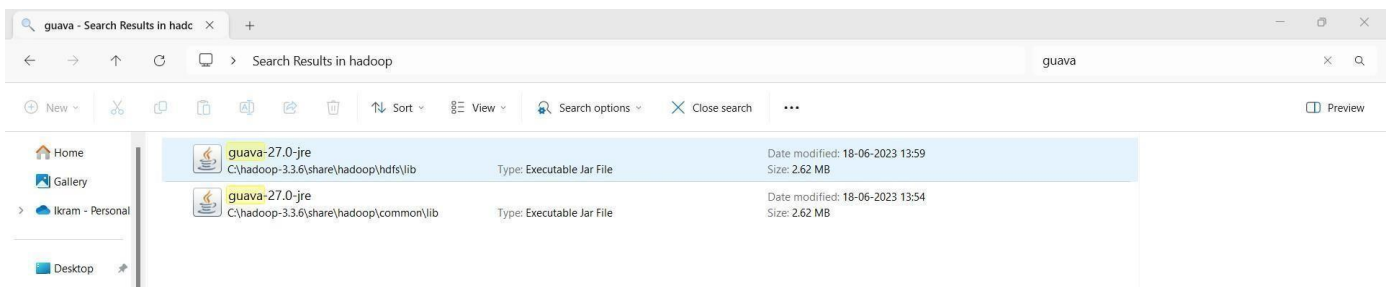
STEP 13: Copy all the JAR Files located in lib folder in Derby folder.



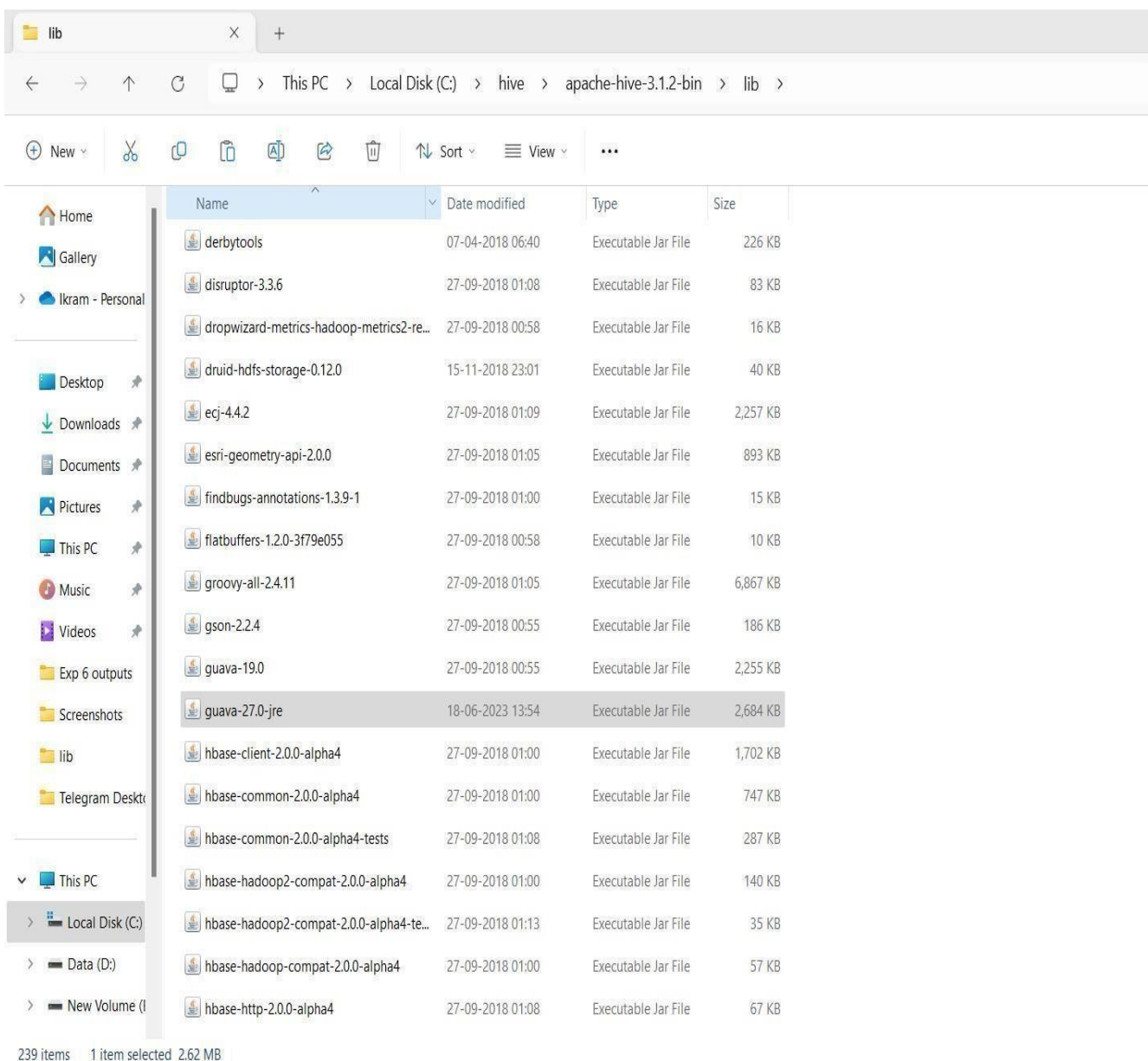
STEP 14: Paste all the JAR Files in lib folder of Hive folder.



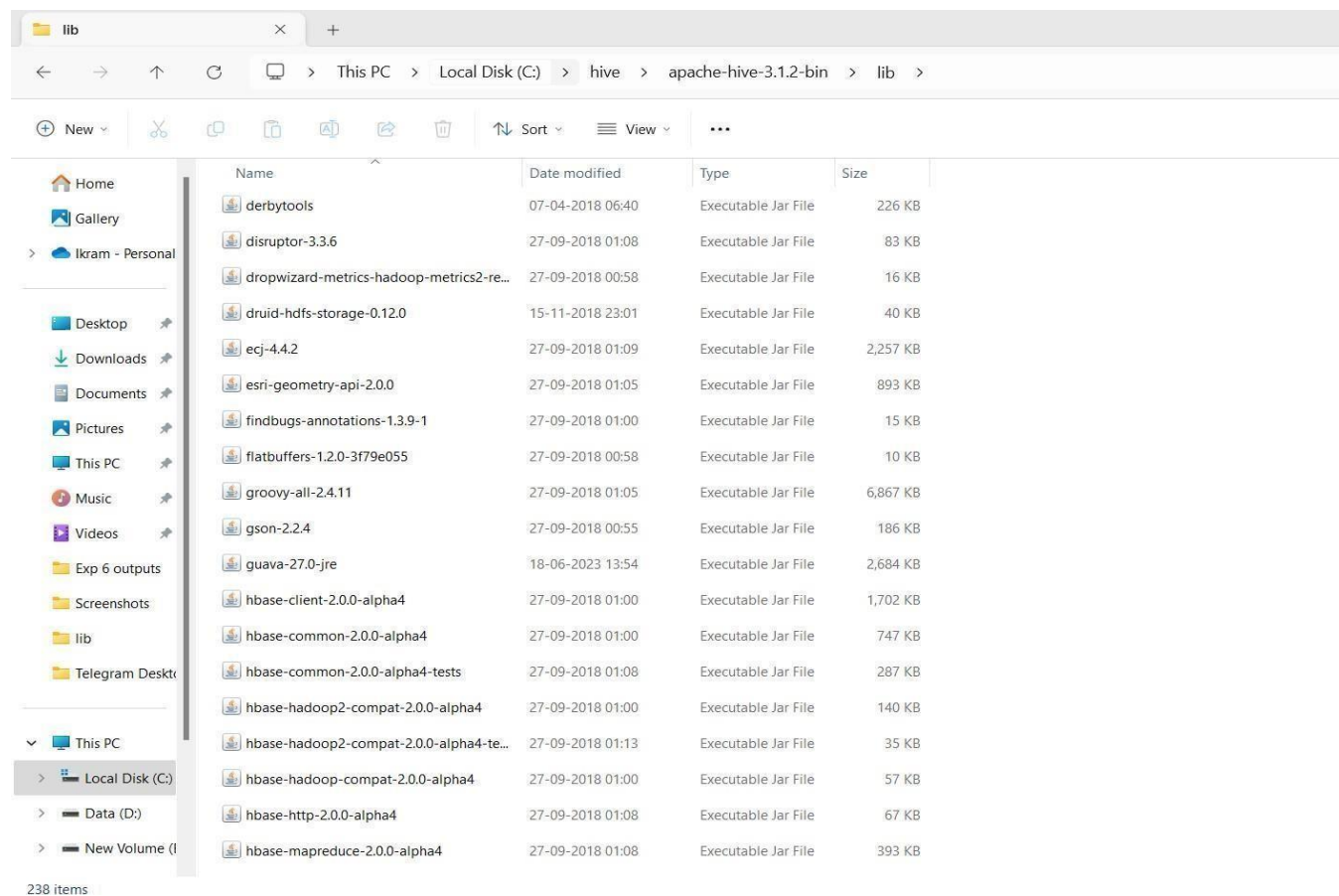
STEP 15: Go to the **Hadoop >> share >> Hadoop** folder. Search for **guava**. Copy the **guava-27.0-jre** file.



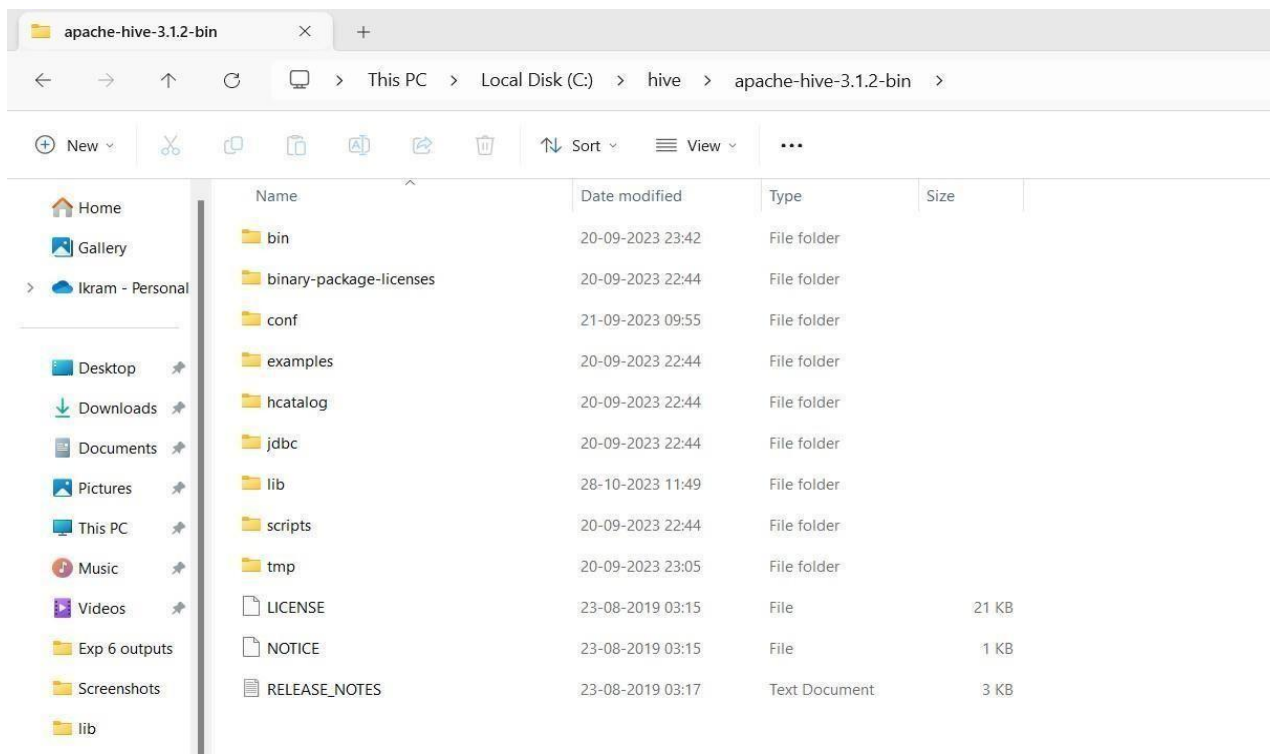
STEP 16: Go to **lib** folder of **Hive** folder and paste the **copied guava** file.



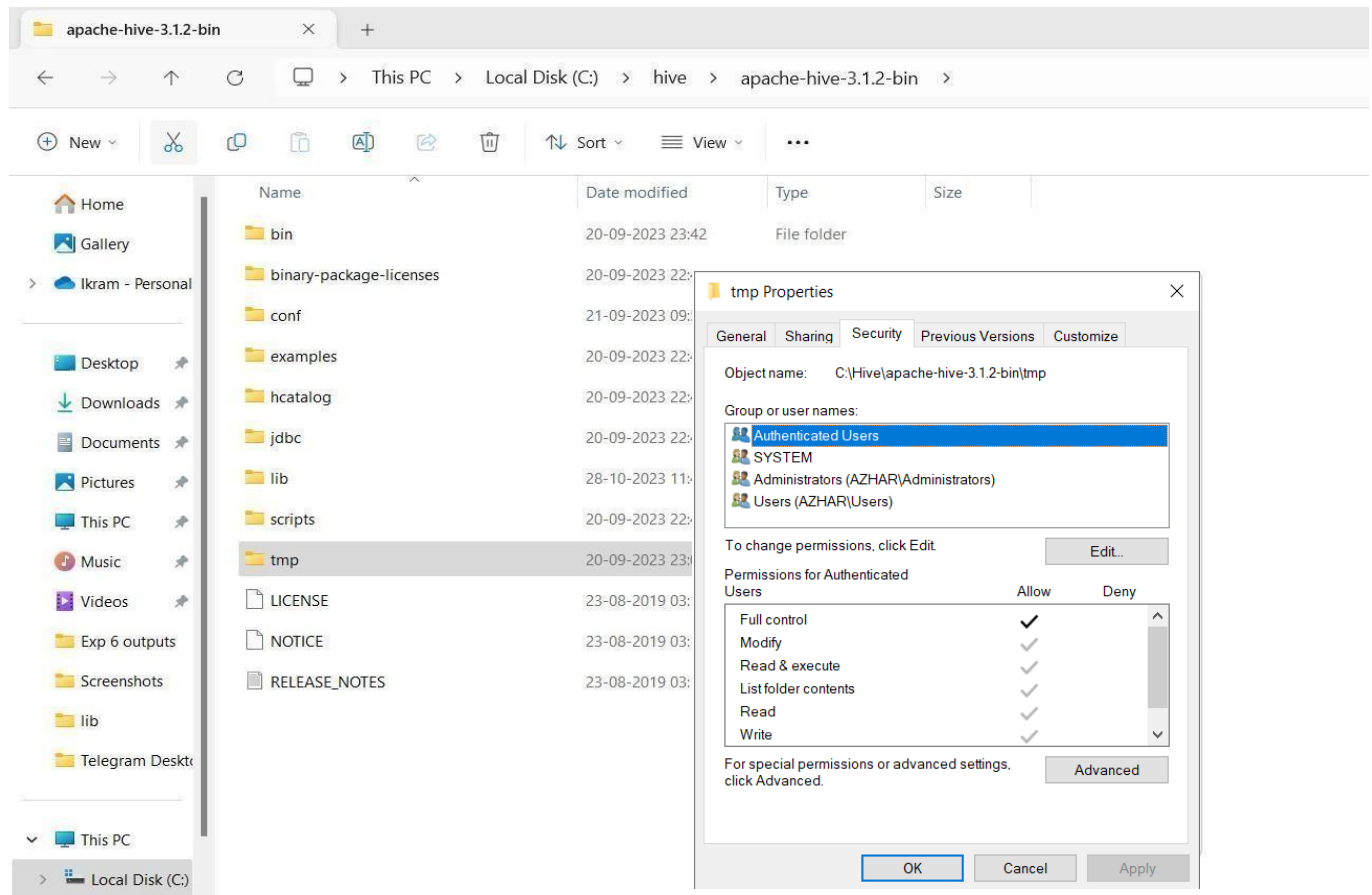
STEP 17: Delete the older version guava file already present in hive folder.



STEP 18: Create a new folder with name tmp in Hive folder.



STEP 19: Go to **Security** in **Properties** of **tmp** file and give **all the permissions** to the **tmp** file.



STEP 20: Go to **conf** folder of **Hive** and create a **New Text file** by name **hive-site.xml** and paste the following code given below.

PROGRAM (hive-site.xml):

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration><property> <name>javax.jdo.option.ConnectionURL</name>
<value>jdbc:derby://localhost:1527/metastore_db;create=true</value>
<description>JDBC connect string for a JDBC metastore</description>
</property>
<property>
  <name>system:java.io.tmpdir</name>
  <value>C:/hive/apache-hive-3.1.2-bin/tmp</value>
</property>
<property>
  <name>system:user.name</name>
```

```

<value>${user.name}</value>
</property>
<property>
<name>javax.jdo.option.ConnectionDriverName</name>
<value>org.apache.derby.jdbc.ClientDriver</value>
<description>Driver class name for a JDBC metastore</description>
</property>
<property>
<name>hive.server2.enable.doAs</name>
<description>Enable user impersonation for HiveServer2</description>
<value>true</value>
</property>
<property>
<name>hive.server2.authentication</name>
<value>NONE</value>
<description> Client authentication types. NONE: no authentication check LDAP: LDAP/AD
based authentication KERBEROS: Kerberos/GSSAPI authentication CUSTOM: Custom
authentication provider (Use with property hive.server2.custom.authentication.class)
</description>
</property>
<property>
<name>datanucleus.autoCreateTables</name>
<value>True</value>
</property>
</configuration>

```

STEP 21: The **bin** file in Hive folder is given for **Linux operating system**. We need to download **bin file** for **Windows operating system** from the official website of Hive.
<https://svn.apache.org/repos/asf/hive/trunk/bin/>

asf - Revision 1912434: /hive/trunk/bin

- [..](#)
- [beeline](#)
- [beeline.cmd](#)
- [derbyserver.cmd](#)
- [ext/](#)
- [hive](#)
- [hive-config.cmd](#)
- [hive-config.sh](#)
- [hive.cmd](#)
- [hiveserver2](#)
- [init-hive-dfs.sh](#)
- [metatool](#)
- [schematool](#)

STEP 22: To install **bin** file for **Windows Operating system** we need to install **wget** from official website <https://eternallybored.org/misc/wget/> according to your system type.

GNU Wget 1.21.4 for Windows x + v

https://eternallybored.org/misc/wget/

Windows binaries of GNU Wget

A command-line utility for retrieving files using HTTP, HTTPS and FTP protocols.

Warning: some antivirus tools recognise wget-1.21.4-win32.zip as potentially dangerous. The file that triggers the warning is wget.exe.debug, which contains debugging symbols for wget.exe, and isn't even executable. If your AV is giving you trouble, and you don't need the documentation or debug symbols, you can download wget.exe directly, or switch to a less broken security product.

All of the binaries are compiled statically, meaning that wget.exe doesn't require any other files to work.

Version	x86	x64	ARM64	Notes
1.21.4	ZIP EXE	ZIP EXE	ZIP EXE	OpenSSL 3.1.0, ZLib 1.2.13, gpgme-1.20.0, pcre2 10.42, libpsl 0.21.2, c-ares 1.19.0, taskbar progressbar, Windows certificate store support, manual
1.21.3	ZIP EXE	ZIP EXE	ZIP EXE	OpenSSL 1.1.1m, ZLib 1.2.11, gpgme-1.17.1, pcre2 10.39, libpsl 0.21.1, c-ares 1.18.1, taskbar progressbar, Windows certificate store support, manual
1.21.2	ZIP EXE	ZIP EXE	-	OpenSSL 1.1.1l, ZLib 1.2.11, gpgme-1.16.0, pcre2 10.38, libpsl 0.21.1, c-ares 1.17.2, taskbar progressbar, Windows certificate store support, manual
1.21.1	ZIP EXE	ZIP EXE	-	OpenSSL 1.1.1k, ZLib 1.2.11, gpgme-1.15.1, pcre2 10.36, libpsl 0.21.1, c-ares 1.17.1, taskbar progressbar, Windows certificate store support, fix for downloading files >2GB, manual
1.20.3	ZIP EXE	ZIP EXE	-	OpenSSL 1.1.1b, ZLib 1.2.11, gpgme-1.13.0, pcre2 10.32, libpsl 0.20.2, taskbar progressbar, Windows certificate store support, manual
1.20	ZIP EXE	ZIP EXE	-	OpenSSL 1.1.1a, ZLib 1.2.11, gpgme-1.12.0, pcre2 10.32, libpsl 0.20.2, taskbar progressbar, Windows certificate store support, manual; XP support dropped
1.19.4	ZIP EXE	ZIP EXE	-	OpenSSL 1.1.0g, ZLib 1.2.11, gpgme-1.10.0, taskbar progressbar, Windows certificate store support, manual

Windows ports

- GiftSide
- NetCat
- PCL Utilities
- GNU Wget

Scripts

- AutoHotKey for (he)chat for VMWare ESXi

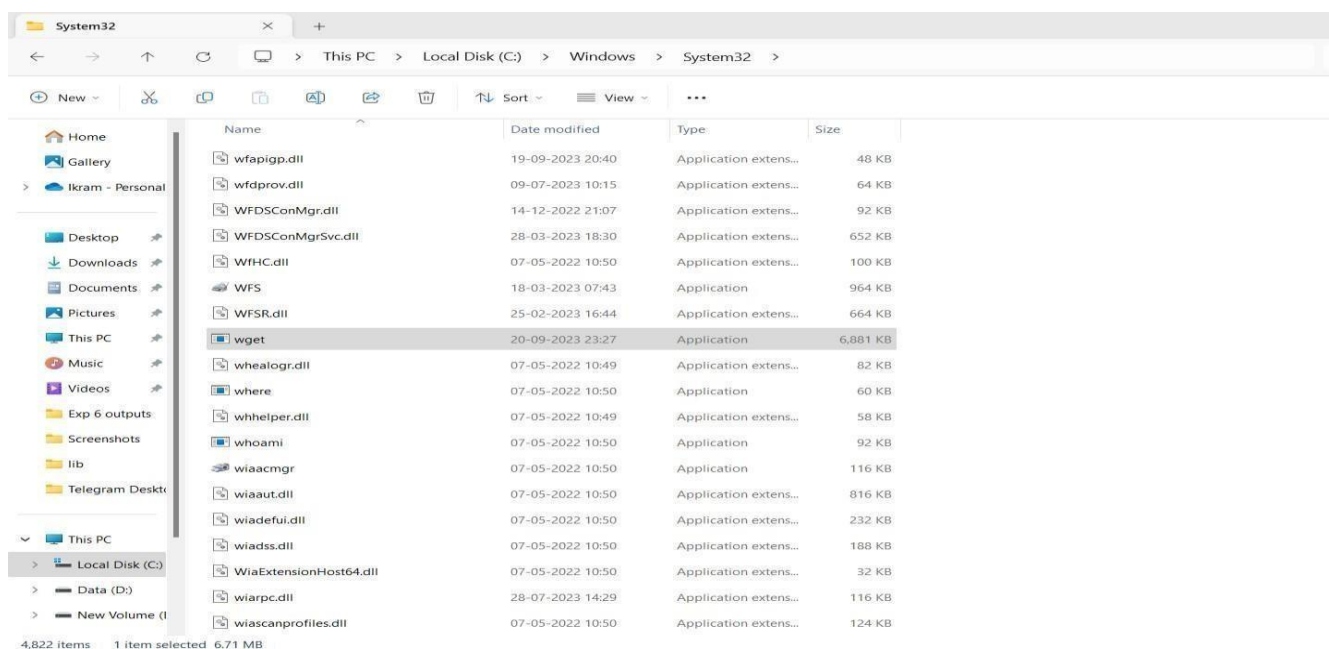
Other things

- GNMP-related stuff

Provided by Jernej Simončič

Donate

STEP 23: After downloading the wget application place it in the C:\Windows\System32



STEP 24: Open Command Prompt and run it as Administrator and run the following command

wget -r -np -nH --cut-dirs=3 -R index.html <https://svn.apache.org/repos/asf/hive/trunk/bin/>

```
Administrator: Command Prom...
Microsoft Windows [Version 10.0.22631.2338]
(c) Microsoft Corporation. All rights reserved.

C:\Users\LEGION>wget -r -np -nH --cut-dirs=3 -R index.html https://svn.apache.org/repos/asf/hive/trunk/bin/
--2023-10-28 12:01:53-- https://svn.apache.org/repos/asf/hive/trunk/bin/
Resolving svn.apache.org (svn.apache.org)... 13.90.137.153
Connecting to svn.apache.org (svn.apache.org)|13.90.137.153|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 760 [text/html]
Saving to: 'trunk/bin/index.html.tmp'

trunk/bin/index.html.tmp      100%[=====>]          760  --.-KB/s   in 0s

2023-10-28 12:01:55 (108 MB/s) - 'trunk/bin/index.html.tmp' saved [760/760]

Loading robots.txt; please ignore errors.
--2023-10-28 12:01:55-- https://svn.apache.org/robots.txt
Reusing existing connection to svn.apache.org:443.
HTTP request sent, awaiting response... 200 OK
Length: 165 [text/plain]
Saving to: 'robots.txt'

robots.txt                    100%[=====>]          165  --.-KB/s   in 0s

2023-10-28 12:01:55 (23.5 MB/s) - 'robots.txt' saved [165/165]

--2023-10-28 12:01:55-- https://svn.apache.org/repos/asf/hive/trunk/bin/beeLine
Reusing existing connection to svn.apache.org:443.
```

```
Administrator: Command Pro x + -
2023-10-28 12:02:12 (124 MB/s) - 'trunk/bin/ext/version.sh' saved [1266/1266]

--2023-10-28 12:02:12-- https://svn.apache.org/repos/asf/hive/trunk/bin/ext/util/execHiveCmd.cmd
Reusing existing connection to svn.apache.org:443.
HTTP request sent, awaiting response... 200 OK
Length: 1078 (1.1K) [text/plain]
Saving to: 'trunk/bin/ext/util/execHiveCmd.cmd'

trunk/bin/ext/util/execHiveCm 100%[=====] 1.05K --.-KB/s in 0s

2023-10-28 12:02:12 (112 MB/s) - 'trunk/bin/ext/util/execHiveCmd.cmd' saved [1078/1078]

--2023-10-28 12:02:12-- https://svn.apache.org/repos/asf/hive/trunk/bin/ext/util/execHiveCmd.sh
Reusing existing connection to svn.apache.org:443.
HTTP request sent, awaiting response... 200 OK
Length: 1149 (1.1K) [text/x-sh]
Saving to: 'trunk/bin/ext/util/execHiveCmd.sh'

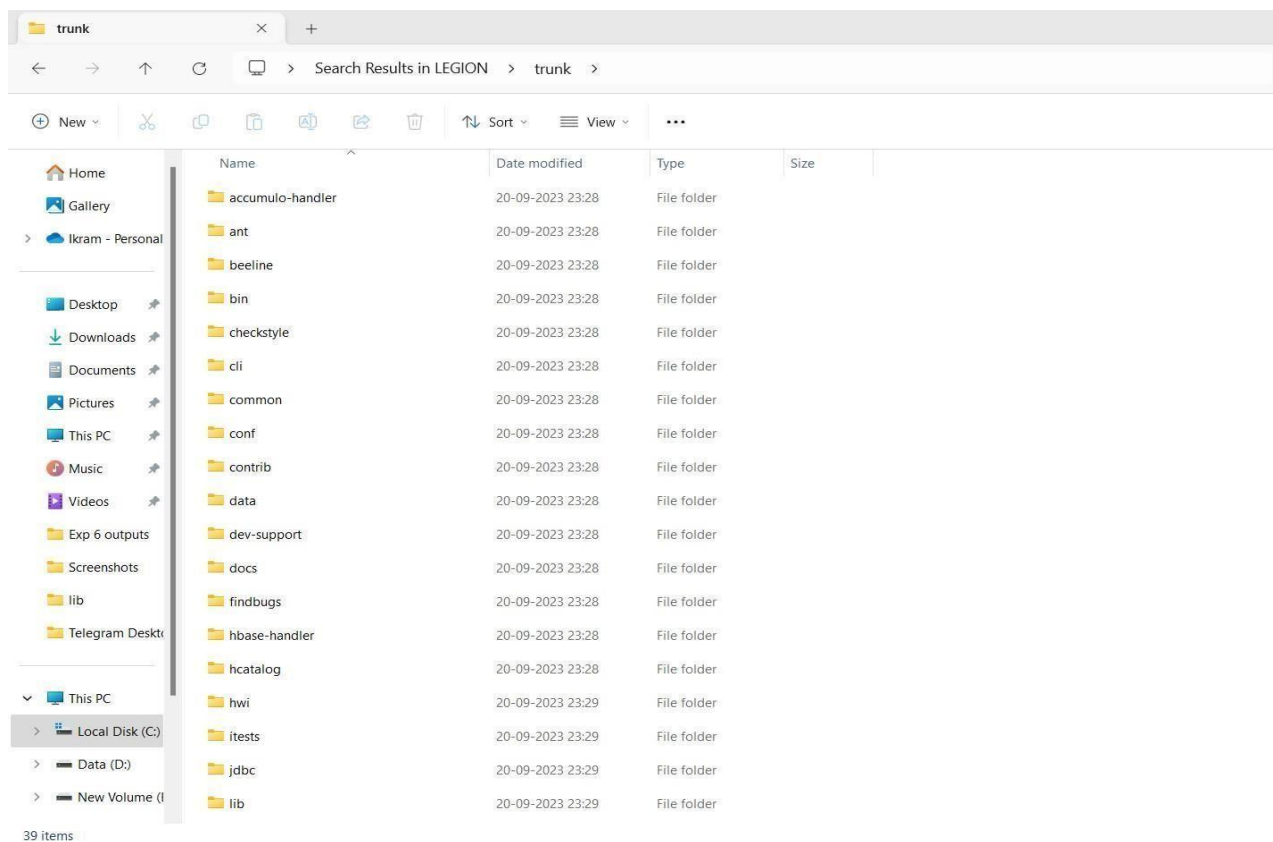
trunk/bin/ext/util/execHiveCm 100%[=====] 1.12K --.-KB/s in 0s

2023-10-28 12:02:12 (125 MB/s) - 'trunk/bin/ext/util/execHiveCmd.sh' saved [1149/1149]

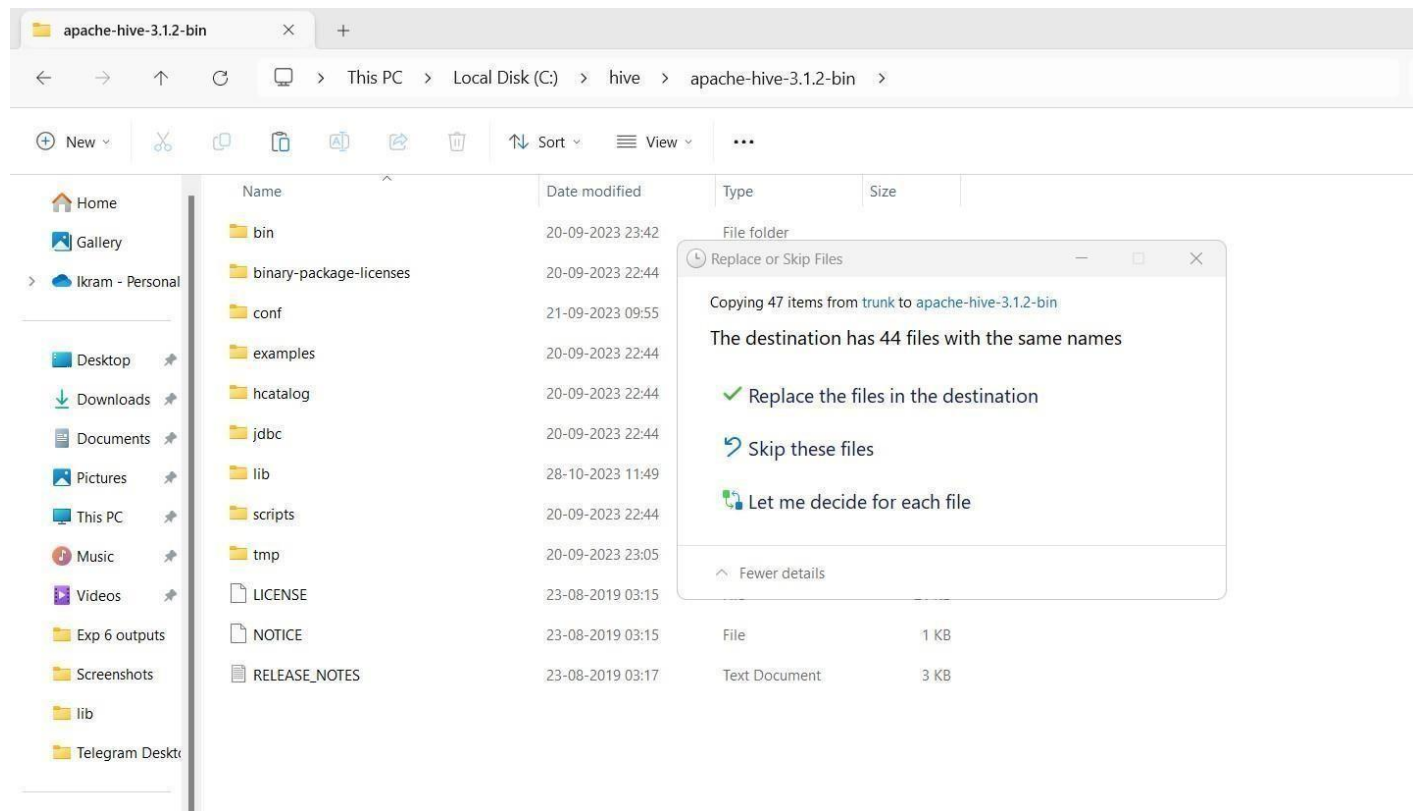
FINISHED --2023-10-28 12:02:12--
Total wall clock time: 19s
Downloaded: 45 files, 76K in 0.03s (2.66 MB/s)

C:\Users\LEGION>
```

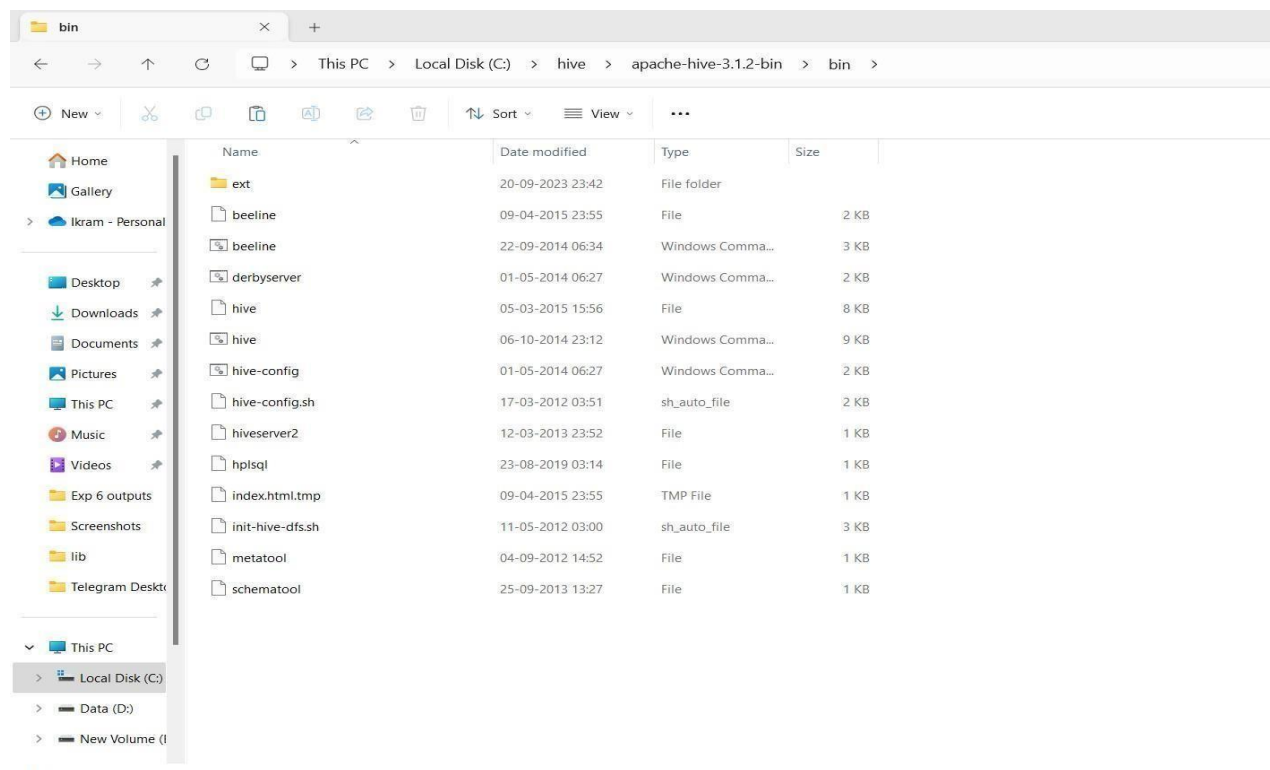
STEP 25: After downloading **search** for the bin file and **copy** the bin file.



STEP 26: Replace this bin file with the bin file present in the Hive directory.



STEP 27: The replaced bin file consists of Windows Community files which are not present in the old bin file.



STEP 28: Run **Command Prompt** as **Administrator** and **run all the demons** of Hadoop.

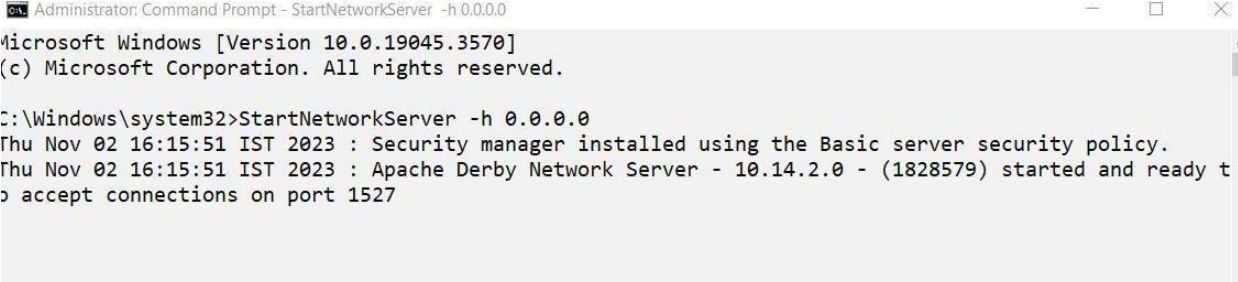
```
C:\Windows\system32>start-all
This script is Deprecated. Instead use start-dfs.cmd and start-yarn.cmd
starting yarn daemons

C:\Windows\system32>jps
2176 Jps
4224 DataNode
9024 NameNode
7924 ResourceManager
3368 NodeManager

C:\Windows\system32>
```

STEP 29: Open new command prompt and run the following command

StartNetworkServer -h 0.0.0.0



```
Administrator: Command Prompt - StartNetworkServer -h 0.0.0.0
Microsoft Windows [Version 10.0.19045.3570]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>StartNetworkServer -h 0.0.0.0
Thu Nov 02 16:15:51 IST 2023 : Security manager installed using the Basic server security policy.
Thu Nov 02 16:15:51 IST 2023 : Apache Derby Network Server - 10.14.2.0 - (1828579) started and ready to
accept connections on port 1527
```

STEP 30: Check whether **NetworkServerControl** is running or not by using **jps** command.

```
C:\Windows\system32>jps
4224 DataNode
9024 NameNode
12916 NetworkServerControl
7924 ResourceManager
3368 NodeManager
5688 Jps

C:\Windows\system32>
```

STEP 31: Go to the bin directory of hive folder and run the following command to create a **metastore** for hive. **hive --service schematool -dbType derby -initSchema**

```

Administrator: Command Prompt
Microsoft Windows [Version 10.0.19045.3570]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>hive --service schematool -dbType derby -initSchema
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/C:/Hive/apache-hive-3.1.2-bin/lib/log4j-slf4j-impl-2.10.0.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/C:/hadoop-3.3.0/share/hadoop/common/lib/slf4j-log4j12-1.7.25.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
2023-11-02T16:33:07,877 INFO [main] org.apache.hadoop.hive.conf.HiveConf - Found configuration file file:/C:/Hive/apache-hive-3.1.2-bin/conf/hive-site.xml
2023-11-02T16:33:08,763 INFO [main] org.apache.hadoop.hive.metastore.tools.HiveSchemaHelper - Metastore connection URL: jdbc:derby://localhost:1527/metastore_db;create=true
Metastore connection URL: jdbc:derby://localhost:1527/metastore_db;create=true
2023-11-02T16:33:08,764 INFO [main] org.apache.hadoop.hive.metastore.tools.HiveSchemaHelper - Metastore Connection Driver : org.apache.derby.jdbc.ClientDriver
Metastore Connection Driver : org.apache.derby.jdbc.ClientDriver
2023-11-02T16:33:08,765 INFO [main] org.apache.hadoop.hive.metastore.tools.HiveSchemaHelper - Metastore connection User: APP
Metastore connection User: APP
Starting metastore schema initialization to 3.1.0
Initialization script hive-schema-3.1.0.derby.sql

```

```

Administrator: Command Prompt

Initialization script completed
schemaTool completed

C:\Windows\System32>

```

STEP 32: In a new **Command Prompt** run **hive** command.

```

Administrator: Command Prompt - hive
Logging initialized using configuration in jar:file:/C:/Hive/apache-hive-3.1.2-bin/lib/hive-common-3.1.2.jar!/hive-log4j2.properties Async: true
2023-11-06T12:55:50,823 INFO [main] org.apache.hadoop.hive.ql.session.SessionState - Created HDFS directory: /tmp/hive/AZHAR
2023-11-06T12:55:50,834 INFO [main] org.apache.hadoop.hive.ql.session.SessionState - Created HDFS directory: /tmp/hive/AZHAR/e4a4daea-a856-4d44-8b5e-5ae33f4c7ea6
2023-11-06T12:55:50,851 INFO [main] org.apache.hadoop.hive.ql.session.SessionState - Created local directory: C:/Users/Dell/AppData/Local/Temp/AZHAR/e4a4daea-a856-4d44-8b5e-5ae33f4c7ea6
2023-11-06T12:55:50,858 INFO [main] org.apache.hadoop.hive.ql.session.SessionState - Created HDFS directory: /tmp/hive/AZHAR/e4a4daea-a856-4d44-8b5e-5ae33f4c7ea6/_tmp_space.db
2023-11-06T12:55:50,891 INFO [main] org.apache.hadoop.hive.conf.HiveConf - Using the default value passed in for log id: e4a4daea-a856-4d44-8b5e-5ae33f4c7ea6
2023-11-06T12:55:50,891 INFO [main] org.apache.hadoop.hive.ql.session.SessionState - Updating thread name to e4a4daea-a856-4d44-8b5e-5ae33f4c7ea6 main
Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. spark, tez) or using Hive 1.X releases.
2023-11-06T12:56:07,193 INFO [e4a4daea-a856-4d44-8b5e-5ae33f4c7ea6 main] CliDriver - Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. spark, tez) or using Hive 1.X releases.
Hive Session ID = 8a31aed3-19cd-4242-8bba-0699cea259a8
2023-11-06T12:56:07,225 INFO [pool-10-thread-1] org.apache.hadoop.hive.ql.session.SessionState - Created HDFS directory: /tmp/hive/AZHAR/8a31aed3-19cd-4242-8bba-0699cea259a8
2023-11-06T12:56:07,230 INFO [pool-10-thread-1] org.apache.hadoop.hive.ql.session.SessionState - Created local directory: C:/Users/Dell/AppData/Local/Temp/AZHAR/8a31aed3-19cd-4242-8bba-0699cea259a8
2023-11-06T12:56:07,238 INFO [pool-10-thread-1] org.apache.hadoop.hive.ql.session.SessionState - Created HDFS directory: /tmp/hive/AZHAR/8a31aed3-19cd-4242-8bba-0699cea259a8/_tmp_space.db
hive>

```

Procedure:

Here's a short algorithm to install Apache Hive (summarized steps in algorithmic format):

Algorithm: Hive Installation

1. Start
2. Check if Java and Hadoop are installed
 - If not, install Java and Hadoop first
3. Download Hive from Apache website using wget
4. Extract the downloaded Hive tarball using tar
5. Move the Hive folder to /usr/local/hive
6. Set environment variables for Hive and Hadoop in .bashrc
 - Update PATH, HIVE_HOME, and HADOOP_HOME
7. Source the .bashrc file to apply changes
8. Create directories for Hive warehouse and temporary files
9. Configure Hive by editing hive-site.xml
10. Initialize Hive Metastore using Derby (default)
11. Start Hive CLI using hive command
12. End

RESULT:

Installation of Apache Hive on a windows operating system is successfully completed and verified.

